VAN LANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



APPLICATION PROGRAMMING
PROJECT OUTLINE

72ITSE41704 · Class group 261_72ITSE41704_01

Title: RESTAURANT TABLE BOOKING AND KITCHEN ORDER SYSTEM

Presented by:
Kieu Bao Giang – 2474802010095 (Leader)
Tang Thoai Lam – 2474802010210 (Member)
Nguyen Nam Khoa – 2474802010194 (Member)
Advisor: Dr. Nguyen Tri Hai

Ho Chi Minh City – 10/10/2026

NON-PLAGIARISM DECLARATION FORM

We, **Kieu Bao Giang** (2474802010095), **Tang Thoai Lam** (2474802010210) and **Nguyen Nam Khoa** (2474802010194), hereby declare that all material in this report, entitled **Restaurant Table Booking and Kitchen Order System**, submitted in partial fulfillment of the requirements of the Faculty of Information Technology, Van Lang School of Technology, Van Lang University, is the result of our own work and does not involve plagiarism.

We confirm that any ideas taken from printed sources (books, magazines) or electronic sources (websites) have been properly credited to the original authors, and that our use of AI tools is declared in Appendix B. We understand that failure to do so may result in rejection or dismissal.

Ho Chi Minh City, Date: 10/10/2026

Signatures of Students

Team leader	*Member*	*Member*
Kieu Bao Giang	**Tang Thoai Lam**	**Nguyen Nam Khoa**

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION 6
1.1 Background and Rationale 6
1.2 Objectives of the Project 7
1.3 Scope of the Project 7
1.4 Research Methodologies 8
1.5 Expected Outcomes 9
1.6 Structure of the Report 9
CHAPTER 2: PROPOSED PROJECT CONTENT 10
2.1 Description of the Project Topic 10
2.2 Proposed Functional Requirements 11
2.3 Proposed Technologies and Tools 16
2.4 Proposed Work Plan 17
CHAPTER 3: EXPECTED RESULTS 19
3.1 Expected Technical Results 19
3.2 Expected Learning Outcomes 20
CHAPTER 4: CONCLUSION 21
APPENDICES 22
Appendix A — Use case specifications 22
Appendix B — Declaration of AI-tool use 27
Appendix C — Design diagrams 30
REFERENCES 55

LIST OF FIGURES

Figure 2.1. Use case diagram 15

Figure C.1. Functional decomposition 30

Figure C.2. Use case diagram 31

Figure C.3. Activity diagram, whole restaurant visit 32

Figure C.4. Activity diagram, UC-03 Create a reservation 33

Figure C.5. Activity diagram, seating a party (UC-04, UC-05, UC-06) 34

Figure C.6. Activity diagram, taking and preparing an order (UC-08 to UC-10) 35

Figure C.7. Activity diagram, billing and payment (UC-12 to UC-14) 36

Figure C.8. Sequence diagram, UC-03 Create reservation (part 1: search) 37

Figure C.9. Sequence diagram, UC-03 Create reservation (part 2: confirm) 38

Figure C.10. Sequence diagram, UC-08 Create order and send to kitchen (part 1: take the order) 39

Figure C.11. Sequence diagram, UC-08 (part 2: send to the kitchen) 40

Figure C.12. Sequence diagram, UC-08 (part 3: kitchen display) 41

Figure C.13. Sequence diagram, UC-13 Split bill 42

Figure C.14. Sequence diagram, UC-14 Record payment and close bill 43

Figure C.15. Class diagram, Table & Reservation and shared classes 44

Figure C.16. Class diagram, Order & Kitchen: menu and orders 45

Figure C.17. Class diagram, Order & Kitchen: kitchen tickets 46

Figure C.18. Class diagram, Billing & Reporting 47

Figure C.19. State diagram, table status lifecycle (BR-05) 48

Figure C.20. ERD, Table & Reservation 49

Figure C.21. ERD, Order & Kitchen: menu and orders 50

Figure C.22. ERD, Order & Kitchen: kitchen tickets 51

Figure C.23. ERD, Billing & Reporting 52

Figure C.24. UI flow, screen navigation 53

Figure C.25. Layered architecture 54

LIST OF TABLES

Table 2.1. Modules and owners 10

Table 2.2. Actors 10

Table 2.3. Functional requirements — Login and Table & Reservation (Kieu Bao Giang) 11

Table 2.4. Functional requirements — Order & Kitchen (Tang Thoai Lam) 12

Table 2.5. Functional requirements — Billing & Reporting (Nguyen Nam Khoa) 12

Table 2.6. Business rules 13

Table 2.7. Use cases 14

Table 2.8. Non-functional requirements 16

Table 2.9. Technologies and tools 17

Table 2.10. Proposed Work Plan 17

Table 3.1. Expected domain classes 19

Table 3.2. Planned screens 19

Table B.1. Declaration of AI-tool use 27

CHAPTER 1: INTRODUCTION

1.1 Background and Rationale

Reasons for selecting the topic

Many small and medium-sized restaurants still run their front of house and kitchen on paper and word of mouth. Reservations are written in a notebook, walk-in guests are queued informally at the door, orders are handwritten and carried to the kitchen, and the bill is assembled by hand at the end of the meal. This causes recurring, visible problems:

- **Front of house:** the same table is promised to two parties, a table is given to walk-in guests shortly before a reservation arrives, and waiting guests are not seated in a fair order.
- **Kitchen:** orders are lost or misread when the restaurant is busy, main courses are prepared before starters, and waiters keep taking orders for dishes that have already run out.
- **Billing:** splitting a bill between guests is done with a calculator and often does not add up, and at the end of a shift nobody can quickly tell how much was taken in cash, by card or by transfer.

The people who suffer are the waiters and hosts who handle the complaints, the kitchen staff who waste food on changed or duplicated orders, the manager who cannot see the state of the restaurant, and the guests who wait longer and receive the wrong dishes.

The reservation side of this problem is documented: Tse and Poon [12] observed a busy restaurant for two years and found that demand regularly exceeded the number of tables, while no-shows, cancellations and walk-ins made it hard to decide how many reservations to accept. The kitchen and billing problems will be confirmed with restaurant staff (Section 1.4).

Importance and practical significance of the topic

One system connects the whole visit — reservation, seating, ordering, cooking and payment — so data entered once is used by every role: fewer double bookings, dishes in the right order, bills that always add up. The topic also fits the course well, with three user roles and rich business rules beyond data entry.

1.2 Objectives of the Project

General objective

Build a web application that lets a small or medium-sized restaurant manage table reservations, walk-in guests, orders, kitchen preparation and billing in one consistent system, with three staff roles.

Specific objectives

All objectives are verified by Week 9, at release v1.0:

1. Implement **15 use cases** end-to-end for three roles (Waiter/Host, Kitchen Staff, Manager).
2. Enforce **13 business rules** in the service layer, each covered by at least one automated unit test.
3. Prevent double booking: **zero** overlapping reservations for the same table, including when two users book at the same moment.
4. Document at least **30 test cases** with expected and actual results (the course minimum for a team of three is 20 [2]).
5. Meet the measurable non-functional requirements in Section 2.2.4, for example an availability search in under 2 seconds and a new order visible in the kitchen within 3 seconds.

1.3 Scope of the Project

Research scope

The project studies the operational workflow of a single small or medium-sized restaurant with one dining room and one kitchen: taking reservations and walk-ins, seating, order taking, kitchen preparation, billing and end-of-shift reporting.

System/application scope

In scope:

- Login and role-based access for three staff roles.
- Tables and sections; reservations (create, modify, cancel, check in, no-show); walk-in waiting list; table status.

- Menu management; order taking; course sequencing; kitchen display; out-of-stock handling.
- Bill generation; even and by-item bill splitting; payments (cash, card, bank transfer); shifts; shift report.

Explicitly out of scope:

- Online booking or ordering by customers; deposits; SMS or email notifications.
- Real payment gateways or card terminals (payments are recorded, not processed); tax invoices.
- Staff account administration in the UI (accounts are seeded); ingredient-level stock; multiple branches.

Prices are integer amounts in VND, VAT included, with no service charge.

Time and organizational scope

Ten weeks, 14 September to 22 November 2026, by a team of three. Each member owns one module end-to-end (requirements, design, code, tests, report sections); the modules share one database and one release schedule.

1.4 Research Methodologies

Information collection

- **Document review:** published research on restaurant reservations, no-shows and walk-ins [12], the course catalogue entry for this topic [2], requirements-engineering guidance [4] and use-case writing guidance [7].
- **Interview:** in Week 5 (12–18 October 2026; the date depends on the members' class schedules), a semi-structured interview with the owner or staff of at least one restaurant in Ho Chi Minh City about reservations, orders, the kitchen and billing, confirming the times used by the business rules (90, 15, 15 and 30 minutes). Findings and any requirement changes will be recorded in /docs and in Chapter 2 of the Final Report.

Observation and practical investigation

If the restaurant agrees, the team will observe the front of house and the kitchen at a busy time and compare what it sees with the use cases.

Data analysis

The information collected is turned into numbered functional requirements, measurable non-functional requirements, business rules and use cases, following ISO/IEC/IEEE 29148 [4] and Cockburn [7].

Comparative and statistical methods

No statistical study is planned; response times are measured on the running system with 50–200 seed records and compared with Table 2.8.

Other methods

Cross-module walkthroughs of the use cases; three iterative releases (v0.1, v0.2, v1.0), each followed by testing.

1.5 Expected Outcomes

Knowledge and skills expected to be gained

Requirements analysis, object-oriented design, layered architecture, automated testing and Git-based teamwork (Section 3.2).

Expected products or deliverables

- **Reports:** this Project Proposal (Week 3) and the Final Project Report (Week 10).
- **Software/application:** a Spring Boot and React web application, tagged v1.0.
- **Database:** a MySQL schema for about 19 domain classes, with realistic seed data.
- **Documentation:** UML diagrams, ERD, wireframes, test-case table and run guide.
- **Prototype/demo system:** a local build that runs by following the README.

1.6 Structure of the Report

The Final Project Report follows the course template [1]: Chapter 1 introduction; Chapter 2 background and requirement analysis; Chapter 3 design, implementation and testing; Chapter 4 results, limitations and future work; appendices with the test log, AI-use declaration and contribution table.

CHAPTER 2: PROPOSED PROJECT CONTENT

2.1 Description of the Project Topic

Overview of the selected topic

The system is catalogue topic **#22, Restaurant Table Booking and Kitchen Order System** [2]. It covers the full journey of a restaurant visit: a guest reserves a table or joins the waiting list, is seated, orders, the kitchen prepares the dishes in the right course order, and the bill is split and paid. It is divided into three modules, one per member, that share one database and one set of users (Table 2.1). Three actors use the system (Table 2.2).

Table 2.1. Modules and owners

Module	Owner	Covers
Table & Reservation	Kieu Bao Giang	Login and access control, tables, reservations, waiting list, table status
Order & Kitchen	Tang Thoai Lam	Menu, orders, course sequencing, kitchen display, out-of-stock
Billing & Reporting	Nguyen Nam Khoa	Bills, splitting, payments, shifts, shift reports

Table 2.2. Actors

Actor	Description
Waiter/Host	Front-of-house staff: reservations, seating, waiting list, orders, bills and payments.
Kitchen Staff	Sees tickets on the kitchen display, updates preparation status, marks dishes out of stock.
Manager	Configures tables and the menu, opens and closes shifts, views reports; can also do everything a Waiter/Host can.

Current problems or practical needs

In the manual process described in Section 1.1, each step depends on the previous one being written down correctly and no one sees the whole picture. The system must therefore check every booking against the table's schedule and cleaning status, send orders to the kitchen in course order, lock what the kitchen has started, and make every split bill add up exactly.

2.2 Proposed Functional Requirements

2.2.1 Main functions and modules

The functional requirements below are numbered FR-01 to FR-28 and grouped by module (Tables 2.3–2.5). Priority is **Must** (needed to pass), **Should** (planned) or **Could** (only if time allows). If time runs short, the Could requirements FR-19 and FR-20 are cut first, then the Should requirements FR-24, FR-10 and FR-06; any cut will be reported in the Final Report. The source drafts for each module are kept in the repository under docs/requirements.

Table 2.3. Functional requirements — Login and Table & Reservation (Kieu Bao Giang)

ID	Requirement	Priority
FR-01	The system shall require every user to log in with a username and password, and shall show only the screens and actions permitted for that user's role.	Must
FR-02	The system shall allow a Manager to create, edit and deactivate a table, with a unique table number, a seat capacity between 1 and 20, and a section. A table that has future reservations cannot be deactivated.	Must
FR-03	The system shall allow a Waiter/Host to search for available tables by date, start time and party size, returning only tables whose capacity is at least the party size and that have no overlapping reservation.	Must
FR-04	The system shall allow a Waiter/Host to create a reservation with guest name, phone number, party size, date, start time and an optional note, and shall reject it if it overlaps another reservation on the same table (BR-01).	Must
FR-05	The system shall allow a Waiter/Host to modify or cancel a reservation before its start time, and to check in a reservation when the guests arrive.	Must
FR-06	The system shall mark a reservation as No-show when the guests have not checked in 15 minutes after the start time, and release the table.	Should
FR-07	The system shall allow a Waiter/Host to add a walk-in party to a waiting list ordered by arrival time, and show each party's position.	Must
FR-08	The system shall allow a Waiter/Host to seat the first waiting party whose size fits a table that has just become Available.	Must
FR-09	The system shall enforce the table status lifecycle (BR-05); a table in Needs Cleaning cannot be assigned until it is marked clean.	Must
FR-10	The system shall display a floor overview of all tables with number, section, capacity and status, colour-coded by status.	Should

Table 2.4. Functional requirements — Order & Kitchen (Tang Thoai Lam)

ID	Requirement	Priority
FR-11	The system shall allow a Waiter/Host to create an order for an Occupied table by choosing items from the menu grouped by category; out-of-stock items are not shown.	Must
FR-12	The system shall allow a Waiter/Host to add items, change the quantity (1–50), remove items and add a note of up to 200 characters to each order line before it is sent to the kitchen.	Must
FR-13	The system shall assign every menu item to a course (Starter, Main, Dessert) and send items to the kitchen in course order (BR-06).	Must
FR-14	When a Waiter/Host sends an order, the system shall create a kitchen ticket containing the sent items and show it on the kitchen display.	Must
FR-15	The system shall allow Kitchen Staff to change the status of each ticket item from New to In Progress to Ready, and shall show the status to the Waiter/Host.	Must
FR-16	The system shall allow a Manager to add, edit and hide menu items, with name, category, course and a price greater than 0 VND. A menu item that appears in any past order can be hidden but not deleted.	Must
FR-17	The system shall allow Kitchen Staff or a Manager to mark a menu item as out of stock or back in stock; an out-of-stock item disappears from every order screen (BR-07).	Must
FR-18	The system shall refuse to modify or cancel an order item once its kitchen ticket item is In Progress (BR-08); new items may still be added and are sent as a new ticket.	Must
FR-19	The system shall allow a Waiter/Host to move an open order to another table.	Could
FR-20	The system shall allow a Waiter/Host to view the orders of a table in the current shift.	Could

Table 2.5. Functional requirements — Billing & Reporting (Nguyen Nam Khoa)

ID	Requirement	Priority
FR-21	The system shall allow a Waiter/Host to generate a bill for an Occupied table from its order, copying each order line with quantity, unit price and line total (BR-09).	Must
FR-22	The system shall display a bill with its lines, grand total, amount paid and amount remaining.	Must
FR-23	The system shall allow a Waiter/Host to split a bill evenly into 2 to 20 shares using the rounding rule BR-10.	Must
FR-24	The system shall allow a Waiter/Host to split a bill by item, assigning every bill line to exactly one share.	Should
FR-25	The system shall allow a Waiter/Host to record one or more payments (cash, card, bank transfer) against a bill or a share, and shall calculate the change for cash payments (BR-11).	Must
FR-26	The system shall close a bill when it is paid in full; closing makes the bill read-only (BR-12) and sets the table to Needs Cleaning (BR-05).	Must

ID	Requirement	Priority
FR-27	The system shall allow a Manager to open a shift and to close it; every payment belongs to the shift that is open when it is recorded (BR-13).	Must
FR-28	The system shall allow a Manager to generate a report for a shift showing the number of closed bills, total revenue, revenue per payment method and number of split bills.	Must

2.2.2 Business rules

These rules are what makes the system more than data entry (Table 2.6). They are implemented in service classes rather than in user-interface event handlers, and each one has automated unit tests. The table lifecycle in BR-05 is drawn as a state diagram in Figure C.19.

Table 2.6. Business rules

ID	Rule	Owner
BR-01	**No double booking.** Two bookings of the same table overlap if startA < endB + turnTime and startB < endA + turnTime, where end = start + diningDuration. Overlapping reservations are rejected; a walk-in cannot be seated at a table whose next reservation starts within one dining duration plus turn-time. Defaults: 90 and 15 minutes, set by the Manager.	Giang
BR-02	**Capacity fit.** A party is only assigned to a table whose capacity is at least the party size.	Giang
BR-03	**Clean before reuse.** A table in Needs Cleaning cannot be assigned until it is marked clean.	Giang
BR-04	**Waiting-list order.** Walk-ins are seated in arrival order; a later party goes first only if no earlier party fits the free table.	Giang
BR-05	**Table lifecycle.** Available → Reserved (from 30 minutes before a confirmed reservation) → Occupied (check-in, also early from Available, or walk-in seated) → Needs Cleaning (automatically when the table's bill is closed, or manually when the table has no open order) → Available (marked clean). A No-show, a cancellation or a change of time or table returns the table from Reserved to Available. No other transition is allowed.	Giang
BR-06	**Course sequencing.** Within an order, items are sent in course order (Starter → Main → Dessert): an item of a later course cannot be sent while an item of an earlier course in the same order is still unsent. The kitchen display lists items by course, then by time sent.	Lam
BR-07	**Out of stock.** An out-of-stock item is removed from every order screen and cannot be added or sent; items already in sent tickets are not affected.	Lam
BR-08	**Lock after cooking starts.** Once a ticket item is In Progress it cannot be modified or cancelled. New items can still be added to the order and are sent as a new ticket.	Lam

ID	Rule	Owner
BR-09	**Bill only when the kitchen is done.** A bill can be generated only when no ticket item of the order is New or In Progress. Generating the bill locks the order; to add items, the unpaid bill must be cancelled first, which is allowed only while no payment has been recorded. A cancelled bill is kept with status Cancelled, so an order has at most one bill that is not cancelled.	Khoa
BR-10	**Even-split rounding.** For N shares, each share is floor(total / N / 1,000) × 1,000 VND and the remainder is added to the first share, so the shares always add up exactly to the bill total. Example: 1,000,000 VND split 3 ways gives 334,000 + 333,000 + 333,000.	Khoa
BR-11	**Valid payments.** A payment amount must be greater than 0. Card and bank-transfer payments cannot exceed the amount remaining; a cash payment may exceed it, and the change is recorded.	Khoa
BR-12	**Closed bills are final.** A bill closes only when payments cover its total. A closed bill cannot be edited, deleted or receive further payments.	Khoa
BR-13	**Shift control.** Payments can be recorded only while a shift is open; only one shift can be open at a time; a shift cannot be closed while it has unpaid bills.	Khoa

2.2.3 Use cases

The requirements are realised by 15 use cases. Figure 2.1 shows them with their primary actors, and Table 2.7 maps each one to its requirements and owner. Their full specifications — actors, preconditions, main and alternative flows, postconditions, and secondary actors — are in Appendix A.

Table 2.7. Use cases

ID	Use case	Primary actor	Requirements	Owner
UC-01	Log in and log out	All roles	FR-01	Giang
UC-02	Manage tables and sections	Manager	FR-02	Giang
UC-03	Create reservation	Waiter/Host	FR-03, FR-04	Giang
UC-04	Modify, cancel or check in a reservation	Waiter/Host	FR-05, FR-06	Giang
UC-05	Manage walk-in waiting list	Waiter/Host	FR-07, FR-08	Giang
UC-06	Update table status	Waiter/Host	FR-09, FR-10	Giang
UC-07	Manage menu	Manager	FR-16	Lam
UC-08	Create order and send to kitchen	Waiter/Host	FR-11 to FR-14, FR-18	Lam
UC-09	Process kitchen ticket	Kitchen Staff	FR-15	Lam
UC-10	Mark menu item out of stock or back in stock	Kitchen Staff, Manager	FR-17	Lam
UC-11	Open and close shift	Manager	FR-27	Khoa
UC-12	Generate bill	Waiter/Host	FR-21, FR-22	Khoa

ID	Use case	Primary actor	Requirements	Owner
UC-13	Split bill	Waiter/Host	FR-23, FR-24	Khoa
UC-14	Record payment and close bill	Waiter/Host	FR-25, FR-26	Khoa
UC-15	Generate shift report	Manager	FR-28	Khoa

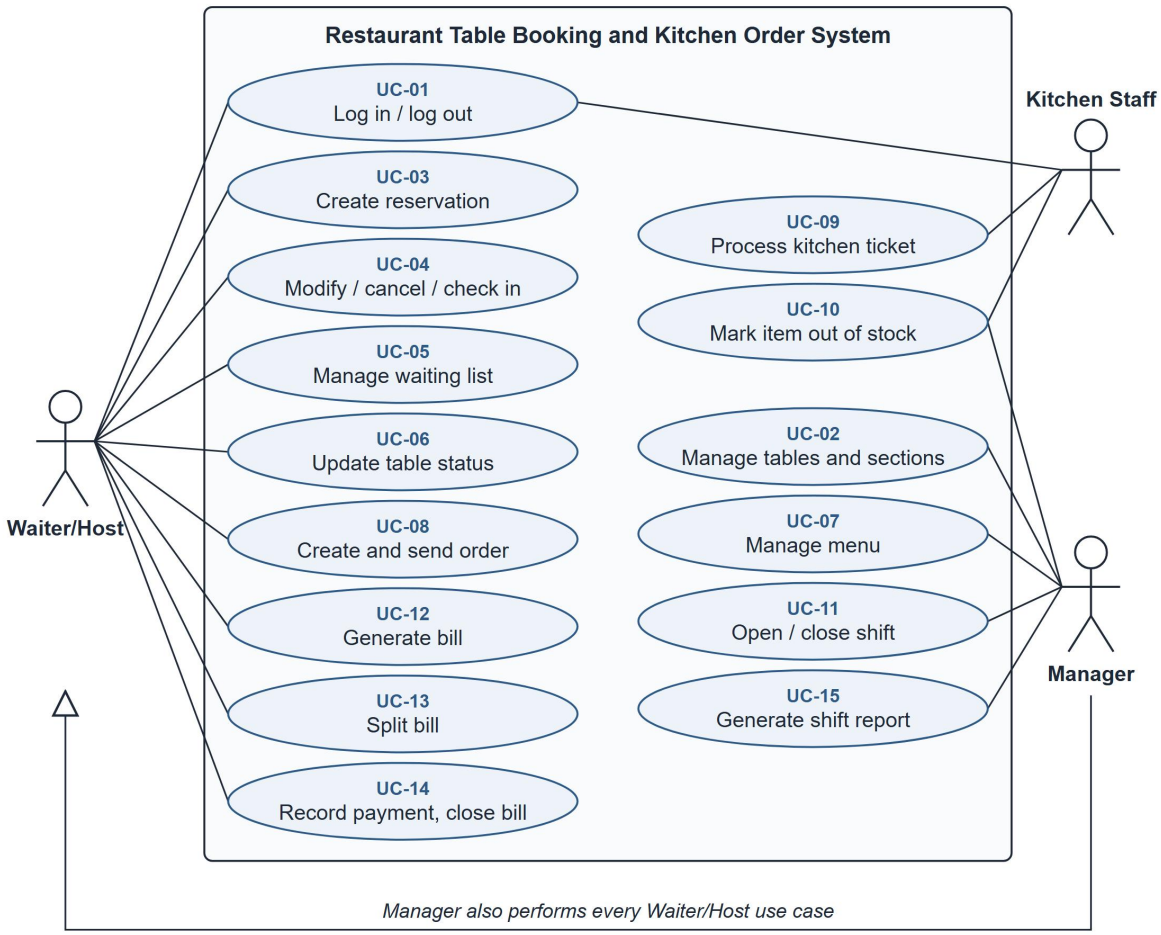


Figure 2.1. Use case diagram

FR-19 and FR-20 are Could requirements and will be added to UC-08 only if time allows.

2.2.4 Expected system features — non-functional requirements

Table 2.8 lists the quality targets the system must meet. Each one is stated so that it can be measured or tested.

Table 2.8. Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	An availability search returns results within 2 seconds for up to 50 tables and 500 reservations per week.
NFR-02	Performance	A sent order appears on the kitchen display within 3 seconds.
NFR-03	Performance	Marking an item out of stock is reflected on every open order screen within 5 seconds, without reloading the page.
NFR-04	Performance	A bill with up to 50 lines is displayed within 2 seconds; a shift report covering up to 300 bills is generated within 3 seconds.
NFR-05	Usability	A Waiter/Host can create a reservation in at most 3 steps, enter a standard 5-item order in at most 60 seconds, and record a single payment in at most 4 steps from the table screen.
NFR-06	Data integrity	When two users confirm the same table for overlapping times at the same moment, exactly one reservation is saved and the other user sees a conflict message. Technique: inside the booking transaction the table row is locked with SELECT … FOR UPDATE (JPA pessimistic write lock) before the overlap check; an automated test submits two bookings at the same moment and checks that exactly one is saved.
NFR-07	Data integrity	Sending an order is atomic: if saving fails, the order stays unsent, no ticket is created, and sending again never creates a duplicate ticket.
NFR-08	Data integrity	Recording the final payment and closing the bill happen in one database transaction; money is stored as integer VND, never as floating-point numbers.
NFR-09	Reliability	All data is stored in MySQL and is unchanged after the application is restarted.
NFR-10	Security	Passwords are stored only as BCrypt hashes; role checks are enforced by the server, and a request outside the user's role receives an "access denied" response.
NFR-11	Validation	Every input is validated on the server; invalid input produces a message next to the field and never causes an unhandled error.
NFR-12	Portability	The application runs on Windows and macOS with Java 17, Node.js 18 or later and MySQL 8, in current versions of Chrome and Edge, and builds from a fresh clone by following the README.

2.3 Proposed Technologies and Tools

The stack is summarised in Table 2.9, grouped as in the template: programming languages, frameworks and platforms, database systems and development tools. Each choice is justified in one sentence. Figure C.25 shows how these technologies form the presentation layer (client and server side), the business layer and the data-access layer.

Table 2.9. Technologies and tools

Category	Technology	Justification
Programming language	Java 17	The team already knows Java and it supports the object-oriented design the course assesses.
Programming language	JavaScript	The language of the React frontend and of every browser.
Framework / platform	Spring Boot 3 [8]	Its controller, service and repository layers give the presentation / business / data-access separation the course requires [3], [6].
Framework / platform	Spring Data JPA (Hibernate)	Maps domain classes to tables and removes boilerplate data-access code, so effort goes into the business rules; generated code will be identified in the report.
Framework / platform	Spring Security, BCrypt	Hashed passwords and server-side role checks for the three roles.
Framework / platform	React 18 [9]	Reusable components suit the role-specific screens (floor overview, order screen, kitchen display).
Framework / platform	Short polling every 2 s	The simplest way to meet NFR-02 and NFR-03; WebSocket is a stretch goal.
Database system	MySQL 8 [10]	A relational database with transactions and unique constraints, which the no-double-booking and payment rules depend on.
Development tool	JUnit 5 [11], Mockito	Unit tests for every business rule in the service layer, independent of the user interface.
Development tool	Git, GitHub	Feature branches, pull requests and release tags v0.1, v0.2 and v1.0, as required by the course.
Development tool	IntelliJ IDEA, VS Code	Java and JavaScript editing, debugging and test running.
Development tool	draw.io or PlantUML	Use-case, class and sequence diagrams in UML 2.5 notation [5] and the ERD.

2.4 Proposed Work Plan

Table 2.10 gives the plan for the ten weeks, with a named person responsible for every task. Week dates run Monday to Sunday; the exact submission deadlines are those published on E-learning/CTE.

Table 2.10. Proposed Work Plan

Week	Tasks	Responsible	Expected output
1 (14–20/9)	Form the team, register the topic, create the repository and README.	Giang (registration, repository); Lam, Khoa (review README)	Registration form; repository with /src, /docs, /tests

Week	Tasks	Responsible	Expected output
2 (21–27/9)	Requirements per module: FR, NFR, business rules, two detailed use cases.	Giang: reservation and login; Lam: order and kitchen; Khoa: billing and reporting	Three requirement drafts in /docs/requirements
3 (28/9–4/10)	Finalise the proposal; interview restaurant staff; review against checklist 12.1.	Giang: Chapters 1 and 4, cover, review; Lam: Chapter 2, UC-07 to UC-10; Khoa: Chapter 3, references, UC-11 to UC-15; Giang and Lam: interview	Project Proposal (PDF + DOCX) — 20%
4 (5–11/10)	Shared layered architecture and ERD; each member draws the class, use-case and sequence diagrams and wireframes of their own module, keeping the drafts and the commit history as evidence.	Giang: architecture, ERD, reservation diagrams; Lam: order and kitchen diagrams; Khoa: billing diagrams	Diagrams and wireframes in /docs
5 (12–18/10)	Restaurant interview (date set around class schedules) and requirement updates from its findings; project skeleton, database schema and login; domain classes, repositories and basic CRUD per module.	All: interview; Giang: skeleton, login, tables and reservations; Lam: menu and orders; Khoa: bills and shifts	Interview notes in /docs; tag v0.1 — application runs and connects to MySQL
6 (19–25/10)	Input validation; shared test-case table; five test cases per module; cross code review.	Khoa: test-case template; each member: own module; reviews Giang→Khoa, Lam→Giang, Khoa→Lam	About 15 documented test cases
7 (26/10–1/11)	Business rules with unit tests, including the concurrent-booking test of NFR-06; Week 7 self-check against the rubric.	Giang: BR-01 to BR-05; Lam: BR-06 to BR-08; Khoa: BR-09 to BR-13	Tag v0.2; self-check in /docs
8 (2–8/11)	Exception handling per module; realistic seed data; security and performance checks.	Lam: seed data (50–200 records); Giang: security check; Khoa: report performance check	Updated diagrams and screenshots
9 (9–15/11)	Feature freeze; end-to-end test of a full visit; run guide; report Chapter 3 per module.	Giang: end-to-end test; Khoa: run guide; each member: own module in Chapter 3	Tag v1.0; draft report
10 (16–22/11)	Complete the report and appendices; repository clean-up; checklist 12.2.	Giang: Chapters 1 and 4; Lam: Chapter 2; Khoa: appendices and test log	Final Project Report (PDF + DOCX) — 80%

CHAPTER 3: EXPECTED RESULTS

3.1 Expected Technical Results

System/application features

- **Front of house:** a colour-coded floor overview, reservation booking with automatic conflict checks, check-in and no-show handling, and a fair walk-in waiting list.
- **Kitchen:** orders sent directly to a kitchen display in course order, preparation status visible to waiters, and out-of-stock dishes removed from ordering immediately.
- **Billing:** bills generated from orders, even and by-item splitting that always adds up, cash, card and bank-transfer payments, and shift reports.
- **Control:** three roles with server-side access checks, and all data kept in MySQL.

Database design

About 19 domain classes are expected (Table 3.1, Figures C.15–C.18); status values and Course are enumerations. TABLE and ORDER are reserved words in SQL, so the tables are named dining_table and customer_order. A kitchen ticket belongs to one order, a bill is generated from one order, and a payment belongs to one bill and one shift.

Table 3.1. Expected domain classes

Module	Main classes
Shared	User (with role)
Table & Reservation	TableSection, DiningTable, Reservation, WaitlistEntry, ReservationSettings
Order & Kitchen	MenuCategory, MenuItem, Order, OrderLine, KitchenTicket, KitchenTicketItem, ItemStatusChange
Billing & Reporting	Bill, BillLine, BillShare, Payment, Shift, ShiftReport

User interfaces

Thirteen screens are planned (Table 3.2); wireframes are drawn in Week 4.

Table 3.2. Planned screens

No.	Screen	Role
1	Login	All

No.	Screen	Role
2	Floor overview	Waiter/Host, Manager
3	Reservation search and booking	Waiter/Host
4	Reservation list (modify, cancel, check in)	Waiter/Host
5	Waiting list	Waiter/Host
6	Table and section management	Manager
7	Menu management	Manager
8	Order taking	Waiter/Host
9	Kitchen display	Kitchen Staff
10	Bill and split	Waiter/Host
11	Payment	Waiter/Host
12	Shift management	Manager
13	Shift report	Manager

Reports or analytical outputs

The shift report (closed bills, revenue in total and per payment method, split bills); a test report with at least 30 documented test cases and unit-test results for all 13 business rules; and measured response times compared with Table 2.8.

3.2 Expected Learning Outcomes

Technical knowledge

Layered architecture with Spring Boot, object-relational mapping with JPA, transactions and constraints in MySQL, React components, and automated unit testing with JUnit.

Practical working experience

Turning a real business process into testable requirements and use cases (CLO1), and designing a layered, object-oriented system documented with UML and an ERD (CLO3).

Teamwork and communication skills

Working as a team through Git branches, pull requests, code reviews and release tags (CLO2), and writing clear technical documentation in English (CLO5).

Problem-solving abilities

Implementing and testing business rules, input validation and exception handling that keep three modules consistent with one another (CLO4).

CHAPTER 4: CONCLUSION

Restaurants that manage reservations, orders and bills on paper lose tables to double bookings, lose orders in the kitchen, and cannot split or reconcile payments reliably. The team proposes a three-role web application, built on a layered Spring Boot and React architecture with a MySQL database, that covers the whole visit from reservation to payment through 15 use cases and 13 business rules enforced in the service layer. The expected outcome is a working, tested system tagged v1.0 in Week 9, with a complete Final Project Report in Week 10.

Main risk. The three modules depend on one another: an order needs an Occupied table, a bill needs a finished order, and closing a bill changes the table status, so a delay or a design mismatch in one module blocks the others.

Mitigation. The shared ERD and the status lifecycles (BR-05, BR-08, BR-09) are agreed in Week 4 before any code is written; the skeleton and database schema are built together in Week 5; and the Billing module starts against sample data so that it does not wait for the other two modules to be finished.

APPENDICES

Appendix A — Use case specifications

Each use case follows the structure recommended by Cockburn [7]: actors, precondition, main flow, alternative flows numbered by the step they branch from, and postcondition.

UC-01: Log in and log out

Actor	Waiter/Host, Kitchen Staff, Manager
Precondition	The user has a seeded account.
Main flow	1. The user opens the application and sees the login screen. 2. The user enters a username and password. 3. The system checks the password against the stored hash. 4. The system opens the home screen for the user's role: floor overview (Waiter/Host, Manager) or kitchen display (Kitchen Staff). 5. When the user clicks Log out, the session ends and the login screen is shown.
Alternative flows	2a. A field is empty: the system shows a validation message. 3a. Wrong username or password: the system shows "Invalid username or password" without saying which one was wrong.
Postcondition	The user is logged in with the functions of their role only.

UC-02: Manage tables and sections

Actor	Manager
Precondition	The Manager is logged in.
Main flow	1. The Manager opens Table management. 2. The Manager adds or edits a table: number, capacity, section. 3. The system validates the input and saves it. 4. The Manager can deactivate a table. 5. The Manager can change the default dining duration and turn-time used by BR-01 (ReservationSettings).
Alternative flows	3a. The table number already exists, or the capacity is outside 1–20: the system shows an error and does not save. 4a. The table has future reservations: the system refuses to deactivate it and shows how many reservations exist.
Postcondition	The table list is updated and used by availability searches.

UC-03: Create reservation

Actor	Waiter/Host
Precondition	The Waiter/Host is logged in; at least one active table exists.

Main flow	1. The Waiter/Host opens the Reservation screen. 2. The Waiter/Host enters the date, start time and party size and clicks Search. 3. The system validates the input and lists the tables that fit the party and have no overlapping reservation (BR-01, BR-02). 4. The Waiter/Host selects a table. 5. The Waiter/Host enters the guest name, phone number and an optional note, and clicks Confirm. 6. The system re-checks the slot, saves the reservation and shows its code.
Alternative flows	3a. Invalid input (date in the past, party size ≤ 0 or larger than any table): a message is shown next to the field. 3b. No table available: the system offers another time or the waiting list (UC-05). 6a. The slot was taken meanwhile by another user: the reservation is rejected with a conflict message and the list is refreshed.
Postcondition	The reservation is Confirmed and the slot is no longer available.

UC-04: Modify, cancel or check in a reservation

Actor	Waiter/Host
Precondition	A Confirmed reservation exists.
Main flow	1. The Waiter/Host opens the reservation list for a date and selects a reservation. 2. Modify: the Waiter/Host changes the time, party size or table; the system re-applies BR-01 and BR-02 and saves. 3. Cancel: the Waiter/Host confirms; the reservation becomes Cancelled and the slot is released. 4. Check in: the guests have arrived; the table becomes Occupied (BR-05).
Alternative flows	2a. The change would overlap another reservation: it is rejected and the original is kept. 2b. The start time has passed: modify and cancel are not allowed. 4a. The table is still Needs Cleaning: check-in is refused until it is marked clean. 4b. More than 15 minutes late: the reservation is already No-show; the system offers to seat the party as a walk-in (UC-05).
Postcondition	The reservation and table status are updated.

UC-05: Manage walk-in waiting list

Actor	Waiter/Host
Precondition	The Waiter/Host is logged in.

Main flow	1. A walk-in party arrives and no suitable table is free. 2. The Waiter/Host enters the name, party size and phone number, and clicks Add to waiting list. 3. The system saves the entry with the arrival time and shows its position. 4. When a table becomes Available, the system highlights the earliest waiting party that fits (BR-04). 5. The Waiter/Host clicks Seat; the table becomes Occupied and the party leaves the list.
Alternative flows	2a. Missing name or party size ≤ 0: validation message. 4a. No waiting party fits: nothing is highlighted. 5a. The party left: the Waiter/Host removes the entry and later parties move up. 5b. The table is Needs Cleaning or has a reservation starting too soon (BR-01): seating is refused with the reason.
Postcondition	The party is waiting or seated.

UC-06: Update table status

Actor	Waiter/Host
Precondition	The Waiter/Host is logged in and sees the floor overview.
Main flow	1. The Waiter/Host selects an Occupied table whose guests have left and clicks Needs cleaning. 2. After cleaning, the Waiter/Host selects the table and clicks Mark clean. 3. The table becomes Available and, if a waiting party fits, it is highlighted (UC-05).
Alternative flows	1a. The table still has an open order or unpaid bill: the system refuses and asks to close the bill first (BR-05). 1b. A transition not allowed by BR-05 is never offered.
Postcondition	The table status follows the lifecycle.

UC-07: Manage menu

Actor	Manager
Precondition	The Manager is logged in.
Main flow	1. The Manager opens Menu management. 2. The Manager adds or edits an item: name, category, course, price, description. 3. The system validates and saves it. 4. The Manager can hide an item.
Alternative flows	3a. Empty name or price ≤ 0: a validation message. 4a. The Manager tries to delete an item used in past orders: the system refuses and offers to hide it instead, so history stays correct.
Postcondition	The menu shown on the order screen is updated.

UC-08: Create order and send to kitchen

Actor	Waiter/Host (secondary: Kitchen Staff)
Precondition	The table is Occupied; the Waiter/Host is logged in.
Main flow	1. The Waiter/Host selects the table. 2. The system shows the menu by category, without out-of-stock items. 3. The Waiter/Host adds items with quantities and notes. 4. The Waiter/Host clicks Send. 5. The system creates a kitchen ticket, respecting course order (BR-06). 6. The ticket appears on the kitchen display. 7. The system confirms that the order was sent.
Alternative flows	3a. An item runs out while it is being added: it is removed with a message before sending (BR-07). 4a. Saving fails: the order stays unsent, the Waiter/Host is told, and retrying creates no duplicate ticket (NFR-07). 7a. The guests order more: new items are sent as a new ticket; items already In Progress cannot be changed (BR-08). 7b. The bill has already been generated: the order is locked (BR-09).
Postcondition	The order is saved and its ticket is on the kitchen display in course order.

UC-09: Process kitchen ticket

Actor	Kitchen Staff (secondary: Waiter/Host)
Precondition	Kitchen Staff is logged in; tickets are waiting.
Main flow	1. Kitchen Staff sees the tickets sorted by course and time sent. 2. Kitchen Staff selects an item and sets it to In Progress; the item is now locked for the waiter (BR-08). 3. When the dish is done, Kitchen Staff sets it to Ready. 4. The Waiter/Host sees that the item is ready.
Alternative flows	3a. A dish is sent back: Kitchen Staff sets it back to In Progress with a reason; the status history is kept (ItemStatusChange, Figure C.3).
Postcondition	Item statuses are updated on the kitchen display and the order screen.

UC-10: Mark menu item out of stock or back in stock

Actor	Kitchen Staff, Manager
Precondition	The user is logged in.
Main flow	1. The user selects a menu item and clicks Out of stock. 2. The system saves the change and within 5 seconds removes the item from every open order screen (NFR-03). 3. Later, the user clicks Back in stock and the item reappears.
Alternative flows	2a. The item is already in sent tickets: those tickets are not affected (BR-07). 2b. A waiter tries to send the item at the same moment: that line is rejected with a message.

Postcondition	Only in-stock items can be ordered.

UC-11: Open and close shift

Actor	Manager
Precondition	The Manager is logged in.
Main flow	1. At the start of service, the Manager clicks Open shift; the system records the start time. 2. At the end of service, the Manager clicks Close shift. 3. The system checks that no bill is unpaid, records the end time and offers the shift report (UC-15).
Alternative flows	1a. A shift is already open: opening is refused (BR-13). 3a. Unpaid bills exist: closing is refused and the unpaid bills are listed.
Postcondition	Exactly one shift is open during service, or none after closing.

UC-12: Generate bill

Actor	Waiter/Host
Precondition	The table is Occupied and has an order; a shift is open.
Main flow	1. The Waiter/Host selects the table and clicks Generate bill. 2. The system checks that no item is New or In Progress (BR-09). 3. The system copies the order lines into bill lines, calculates the total and locks the order. 4. The bill is displayed with total, paid and remaining amounts.
Alternative flows	1a. The table has no order: the system refuses. 2a. Items are still being prepared: the system refuses and lists them. 4a. The guests want to order more: the Waiter/Host cancels the unpaid bill (it is kept with status Cancelled), which is allowed only if no payment has been recorded, and the order is unlocked.
Postcondition	An open bill exists for the table.

UC-13: Split bill

Actor	Waiter/Host
Precondition	The bill is open and has no payments yet.
Main flow	1. The Waiter/Host opens the bill and clicks Split. 2. Evenly: the Waiter/Host enters the number of shares; the system calculates the shares with BR-10. By item: the Waiter/Host assigns each bill line to a share. 3. The system shows each share's amount; the shares always add up to the bill total.
Alternative flows	1a. A payment was already recorded: splitting is refused. 2a. The number of shares is outside 2–20: a validation message. 2b. A line is unassigned or assigned twice: the split cannot be confirmed.
Postcondition	The bill is divided into shares that can be paid separately.

UC-14: Record payment and close bill

Actor	Waiter/Host
Precondition	The bill (or share) is open; a shift is open.
Main flow	1. The Waiter/Host opens the bill or share. 2. The Waiter/Host selects the method (cash, card, bank transfer) and enters the amount. 3. The system validates it (BR-11) and, for cash, shows the change. 4. The system records the payment. 5. When payments cover the total, the system closes the bill in the same transaction, makes it read-only (BR-12) and sets the table to Needs Cleaning (BR-05).
Alternative flows	1a. The bill is already closed: no payment can be added. 1b. No shift is open: payment is refused (BR-13). 3a. Amount ≤ 0, or a card or transfer amount larger than the remaining amount: an error message. 4a. The database save fails: nothing is recorded and the user can retry (NFR-08). 5a. The amount is less than the remaining amount: the payment is recorded and the bill stays open with the new remaining amount.
Postcondition	Payments are stored; a fully paid bill is closed.

UC-15: Generate shift report

Actor	Manager
Precondition	The Manager is logged in; at least one shift exists.
Main flow	1. The Manager opens Shift reports. 2. The Manager selects a shift. 3. The system collects the shift's bills and payments. 4. The system shows the number of closed bills, total revenue, revenue per payment method and number of split bills.
Alternative flows	2a. The selected shift is still open: the report is labelled provisional. 4a. The shift has no bills: the report shows zero revenue.
Postcondition	The Manager sees the figures for the shift.

Appendix B — Declaration of AI-tool use

Table B.1. Declaration of AI-tool use

Tool	What it was used for	Which part of the work	How the output was verified
Claude (Anthropic), via Claude Code	Analysing the course documents; proposing the ten-week timeline and task split	Planning (Week 1)	The team compared the proposed timeline and task split with the course Guidelines and rubric, agreed

Tool	What it was used for	Which part of the work	How the output was verified
			the module assignments in a group meeting and adjusted the plan to each member's responsibilities. Submission deadlines were checked directly on E-learning.
Claude (Anthropic), via Claude Code	Drafting requirements, business rules and use cases for the Table & Reservation module	Chapter 2; Appendix A (UC-01 to UC-06)	Kieu Bao Giang read every requirement, business rule and use case of the module and compared them with the course Guidelines and the Week 2 task list. The drafts were reviewed in several rounds and only agreed changes were kept, for example: a cancelled or moved reservation returns the table to Available (BR-05), the Manager can no longer override table status, and early check-in was added to the table lifecycle. The BR-01 overlap rule was recalculated by hand with sample times. The module draft was committed from Giang's own GitHub account.
Claude (Anthropic), via Claude Code	Lam wrote the original Order & Kitchen draft in Vietnamese (10 FRs, 5 NFRs, BR-06 to BR-08, UC-08 and UC-09) and Khoa wrote the original Billing draft (5 FRs, 5 NFRs, UC-14 and UC-15). Claude translated Lam's draft into English and made its NFRs measurable; for Billing it proposed measurable NFRs, business rules BR-09 to BR-13 and the expansion to FR-21 to FR-28; it also drafted the specifications of UC-07 and UC-10 to UC-13.	Chapter 2; Appendix A (UC-07 to UC-15)	Lam compared the English Order & Kitchen text with the original Vietnamese draft to check meaning and terminology. Khoa checked that the Billing NFRs are measurable and the business rules testable and consistent with the use cases. Both re-read their module against the team proposal and the Guidelines checklist and corrected inconsistencies before the team review.
Claude (Anthropic), via Claude Code	Drafting and assembling the proposal from the module drafts and formatting it in this template	Chapters 1–4	All three members read the assembled proposal in full against their own module drafts and the course

Tool	What it was used for	Which part of the work	How the output was verified
			Guidelines, checked that the requirements, business rules and use cases were consistent and that numbering, captions and references were correct, and corrected the inconsistencies found before submission.
Claude (Anthropic), via Claude Code	Searching for a published source on restaurant no-shows and walk-ins	Section 1.1; reference [12]	Title, authors, journal, volume, pages and DOI checked against the published PDF of the article
Claude (Anthropic), via Claude Code	Generating the diagrams of Figure 2.1 and Appendix C (Mermaid) from the requirements, to cross-check the members' own diagrams and give them a readable layout	Figure 2.1; Appendix C	Compared element by element with the diagrams each member drew for their own module (Giang, Lam, Khoa): they match. The members' own diagrams are kept in the repository as evidence.

From Week 4 onward, in line with the course AI policy, each member produces the requirement changes, diagrams and design models of their own module (use-case, class and sequence diagrams, their part of the ERD and the wireframes) and keeps the drafts and the commit history as evidence. AI tools are used only for outlining, editing and formatting; every such use is declared in the Final Report, and each member is ready to explain any requirement, rule or use case of their module in an oral check.

We declare that we understand every part of the submitted work and can explain it on request.

Appendix C — Design diagrams

These diagrams were generated in Mermaid with AI support (Table B.1) from the requirements in Chapter 2 and used to cross-check the diagrams that each member drew for their own module; the members' own diagrams are kept in the repository. They show the functional decomposition (Figure C.1), the use cases (Figure C.2), the business workflows (Figures C.3–C.7), the main interactions (Figures C.8–C.14), the domain classes by module (Figures C.15–C.18), the table lifecycle of BR-05 (Figure C.19), the database tables (Figures C.20–C.23), the screen navigation (Figure C.24) and the layered architecture (Figure C.25). They will be kept consistent with the code, as the course requires.

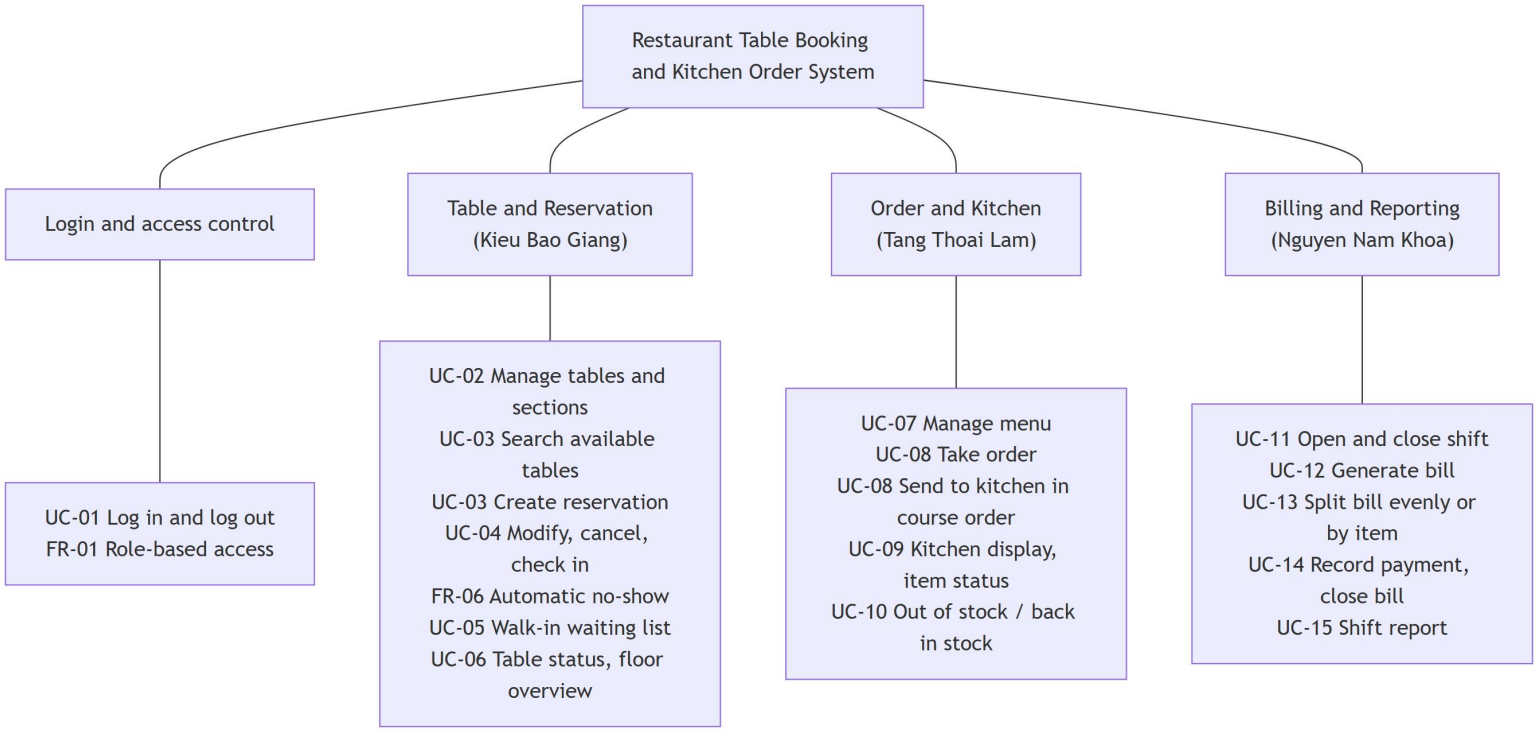


Figure C.1. Functional decomposition

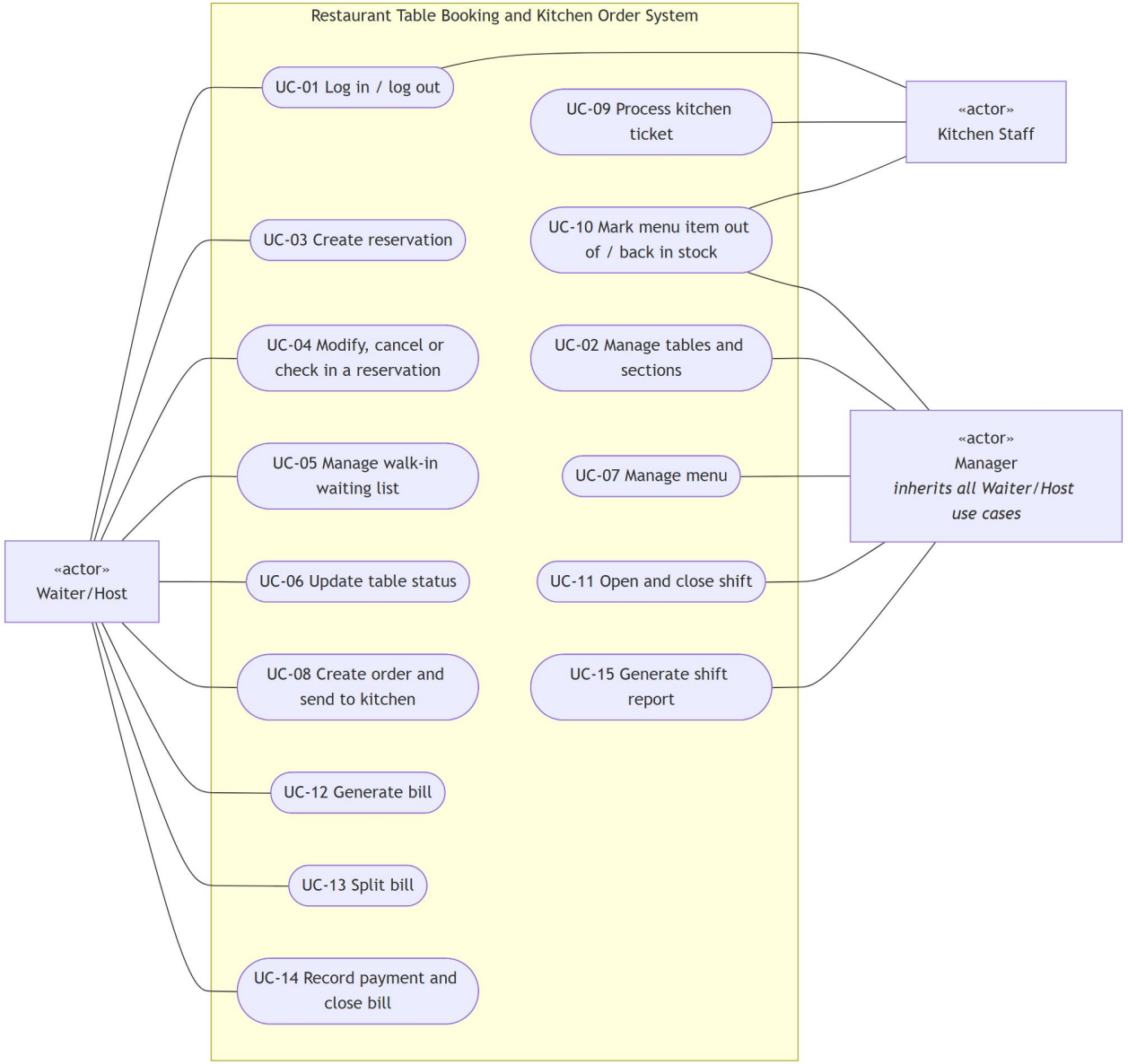


Figure C.2. Use case diagram

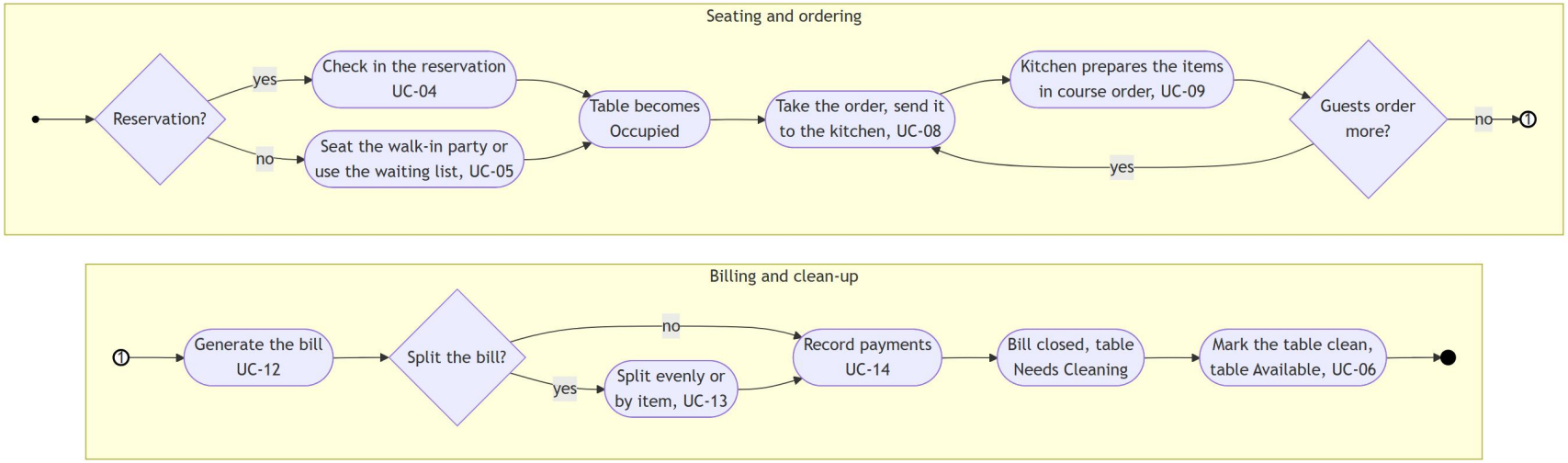


Figure C.3. Activity diagram, whole restaurant visit

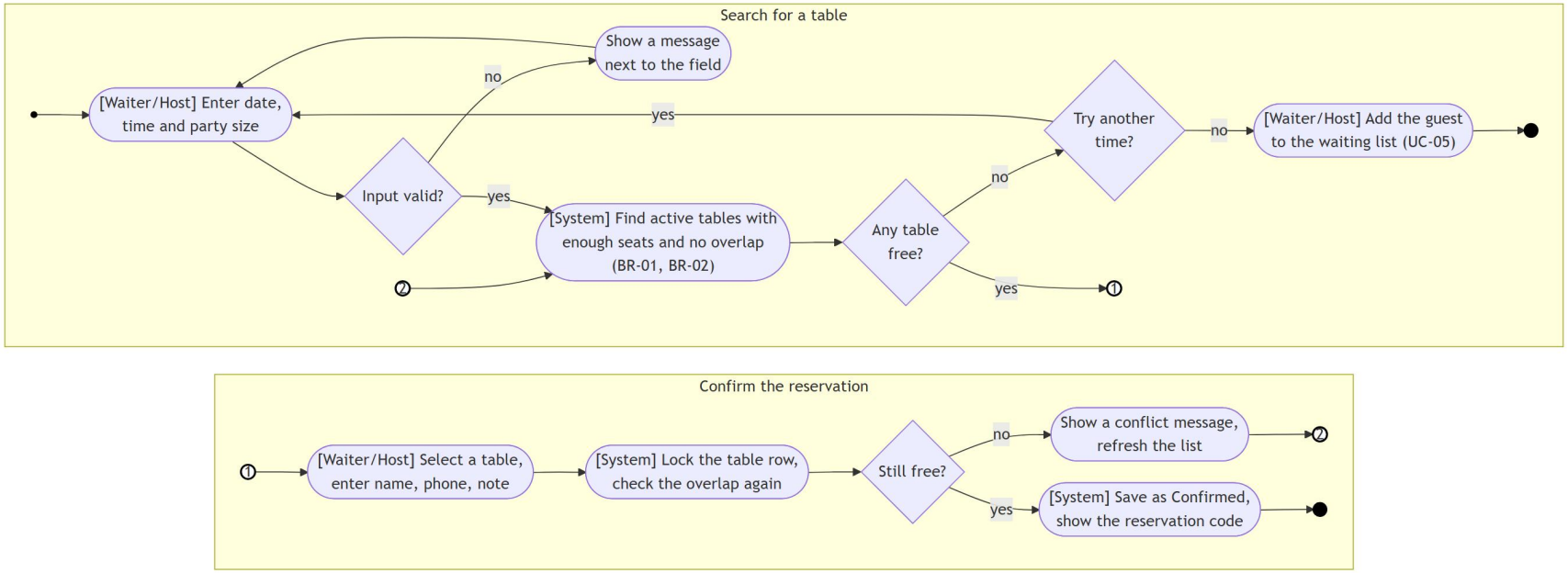


Figure C.4. Activity diagram, UC-03 Create a reservation

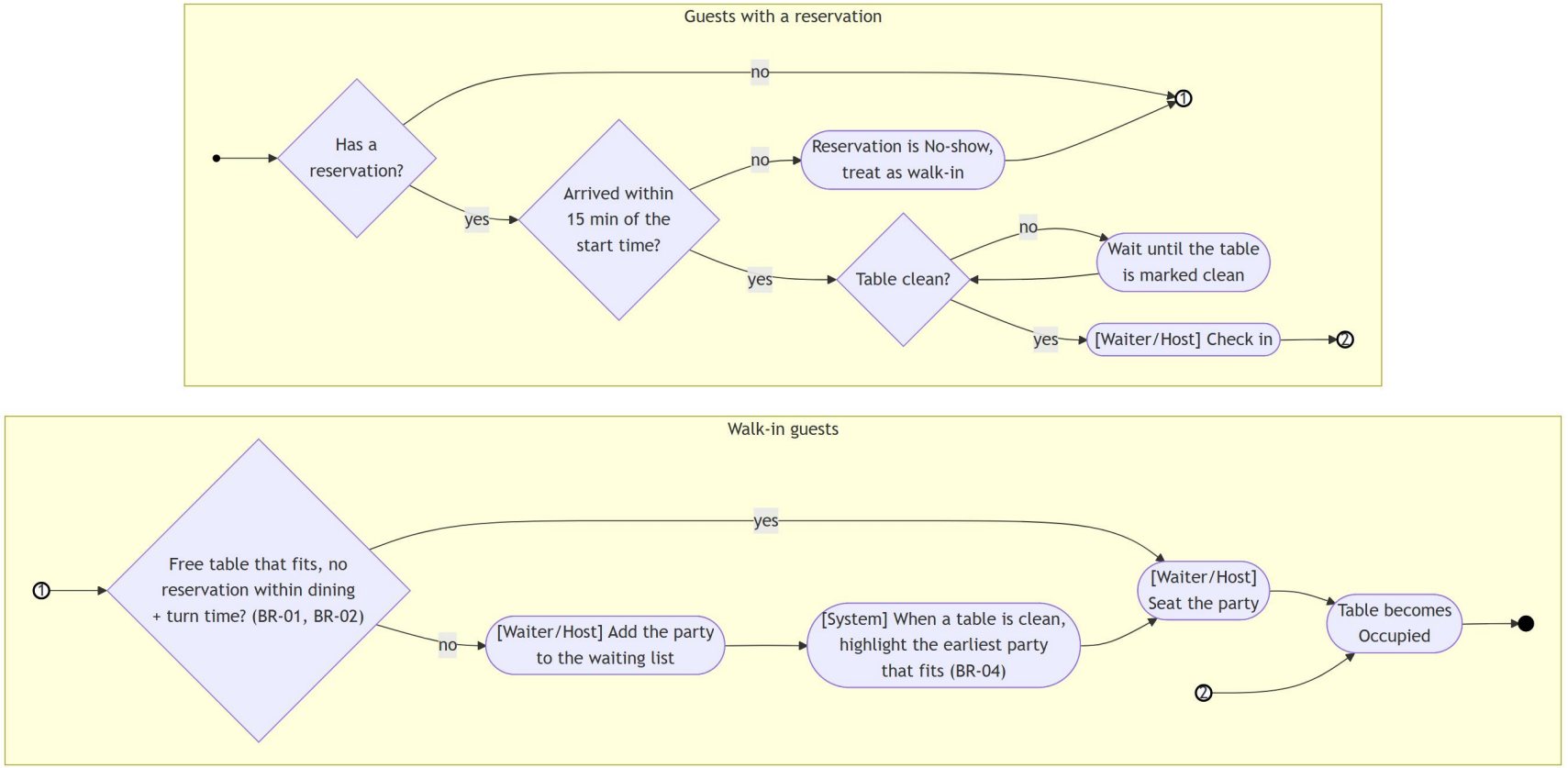


Figure C.5. Activity diagram, seating a party (UC-04, UC-05, UC-06)

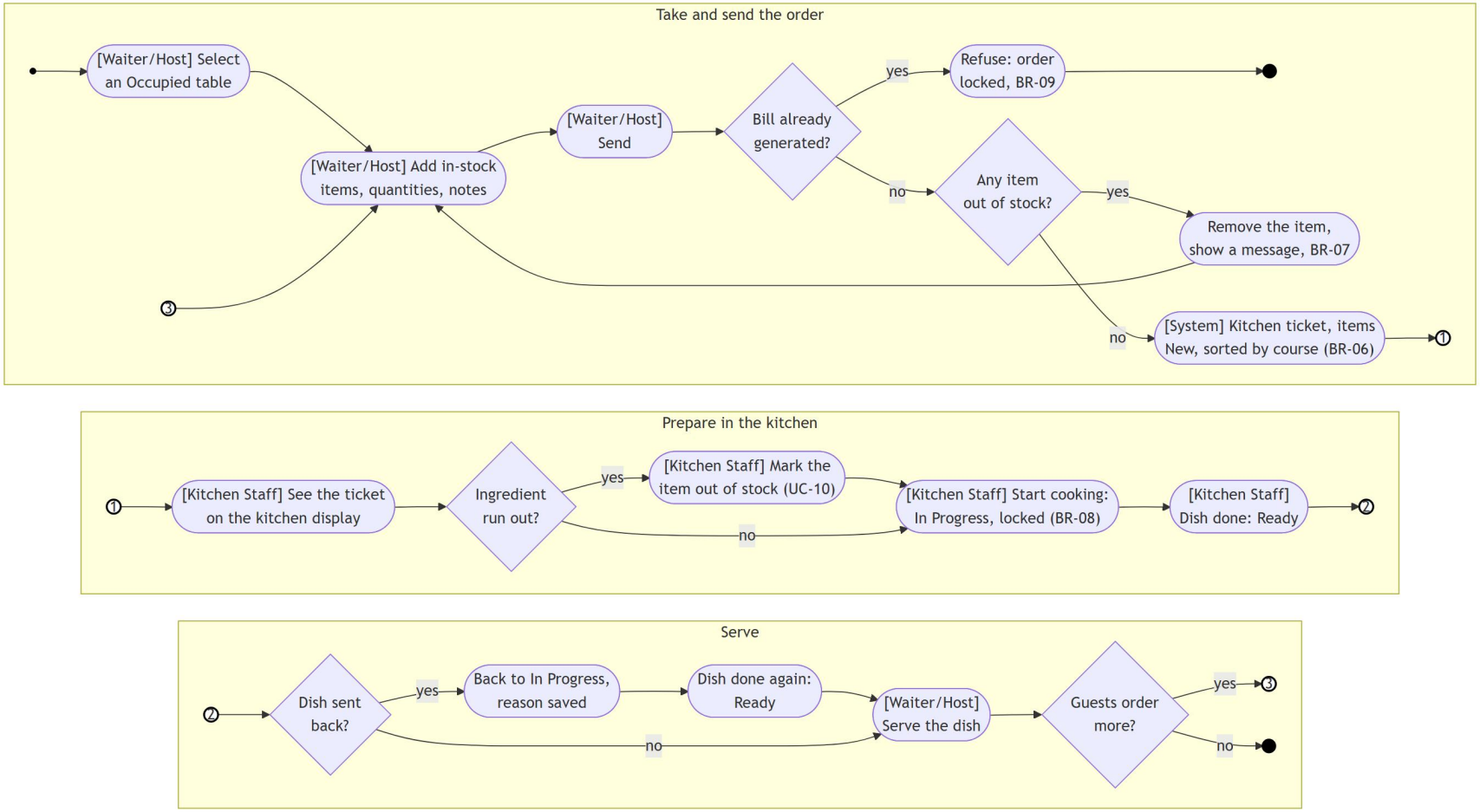


Figure C.6. Activity diagram, taking and preparing an order (UC-08 to UC-10)

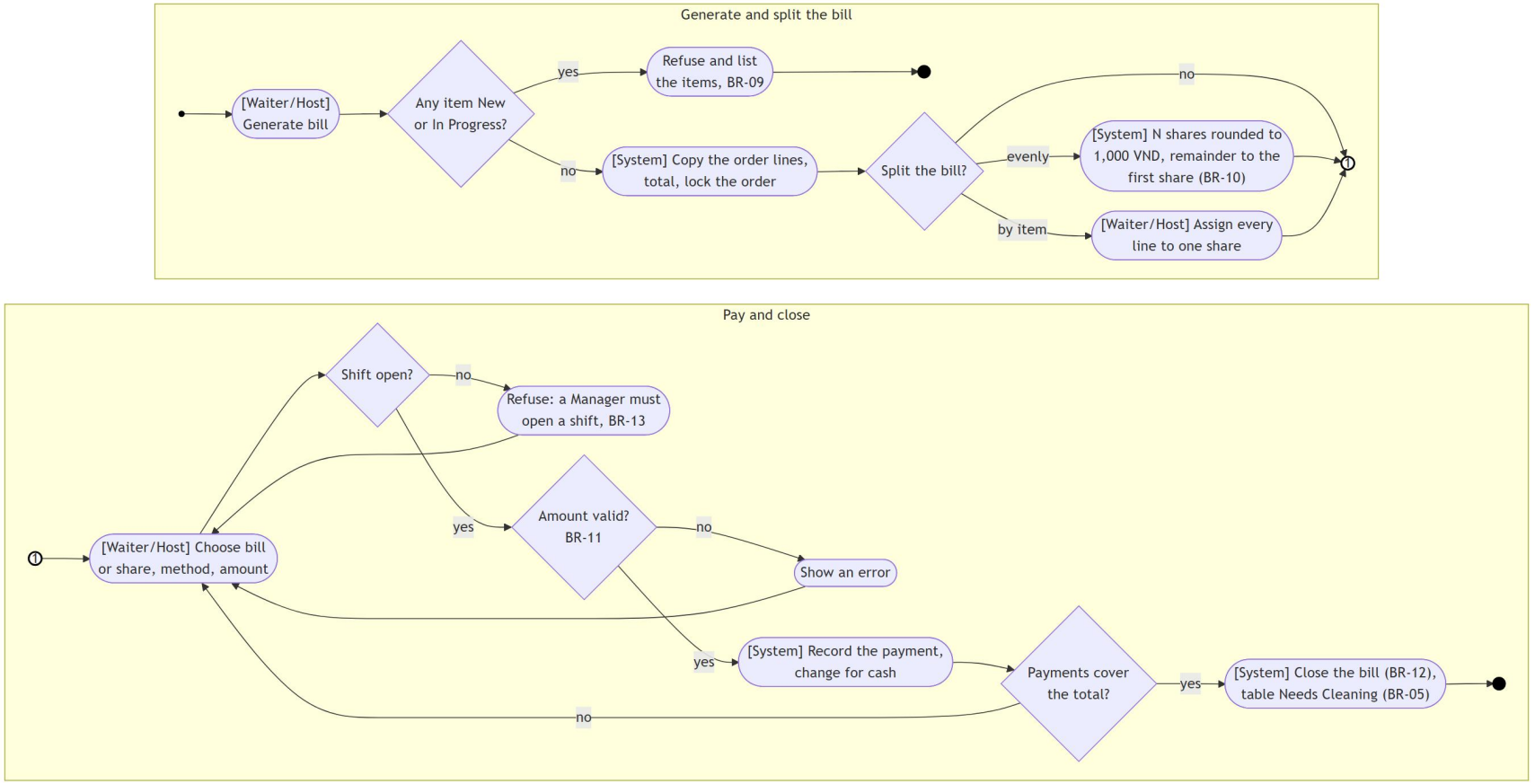


Figure C.7. Activity diagram, billing and payment (UC-12 to UC-14)

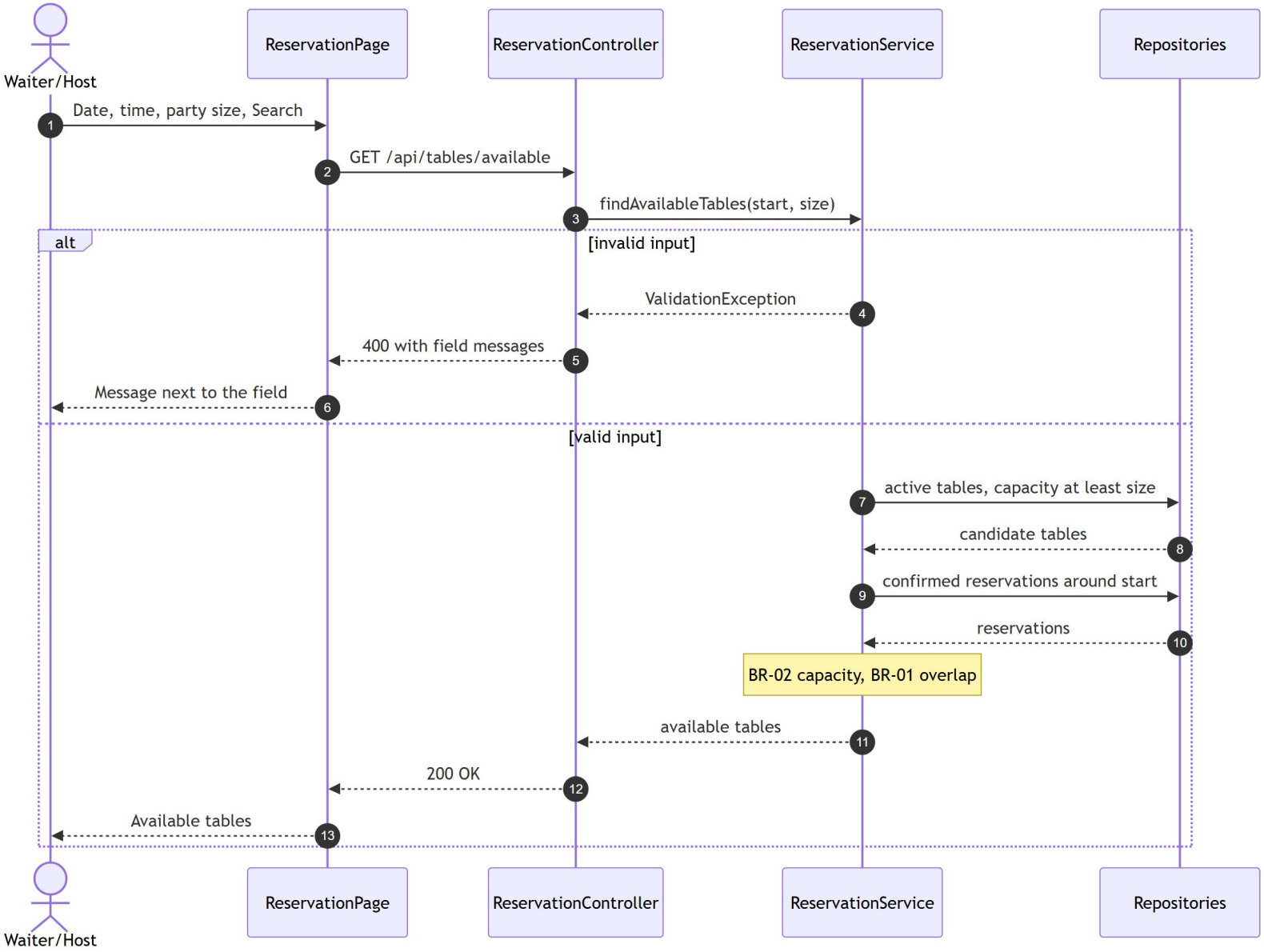


Figure C.8. Sequence diagram, UC-03 Create reservation (part 1: search)

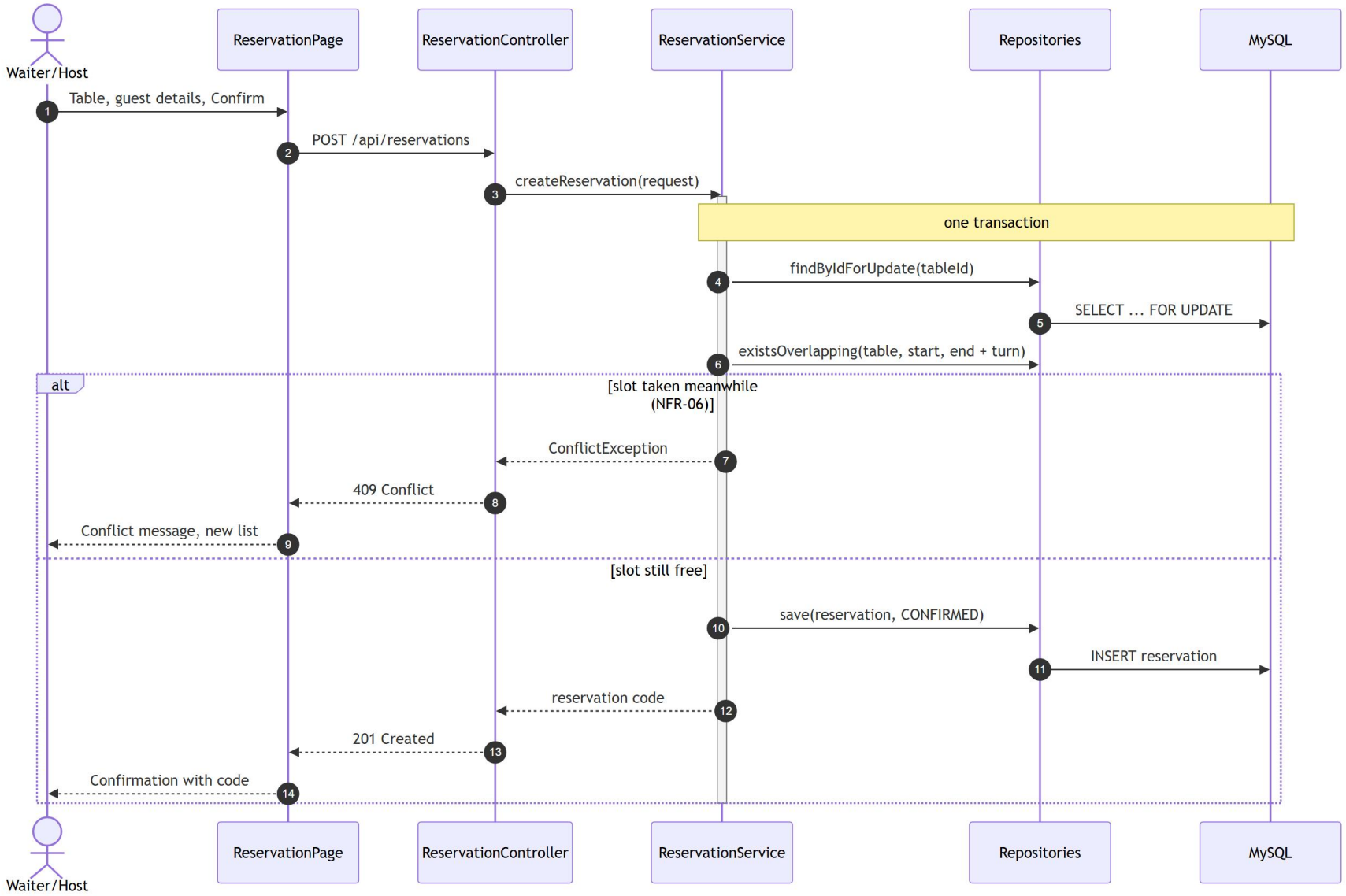


Figure C.9. Sequence diagram, UC-03 Create reservation (part 2: confirm)

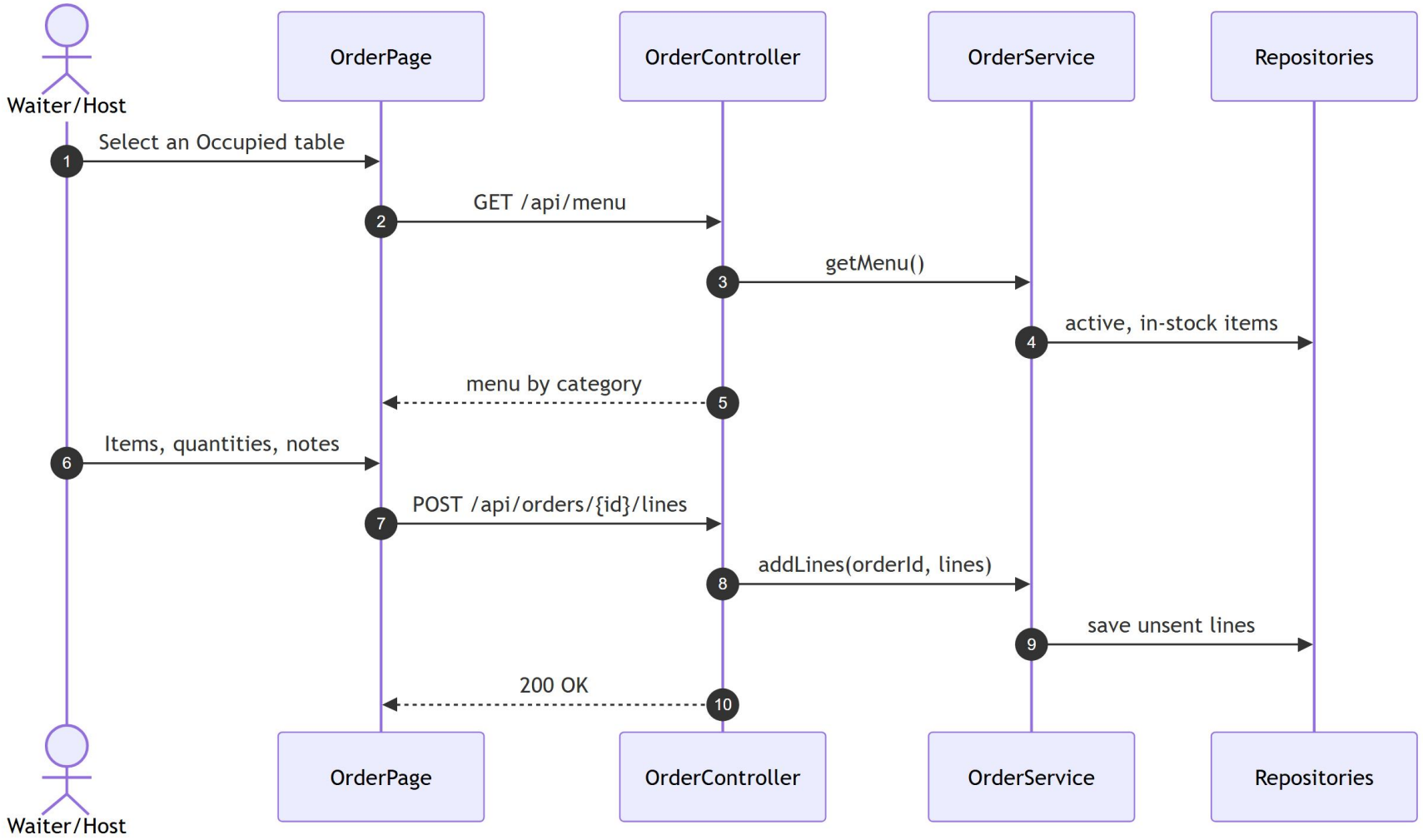


Figure C.10. Sequence diagram, UC-08 Create order and send to kitchen (part 1: take the order)

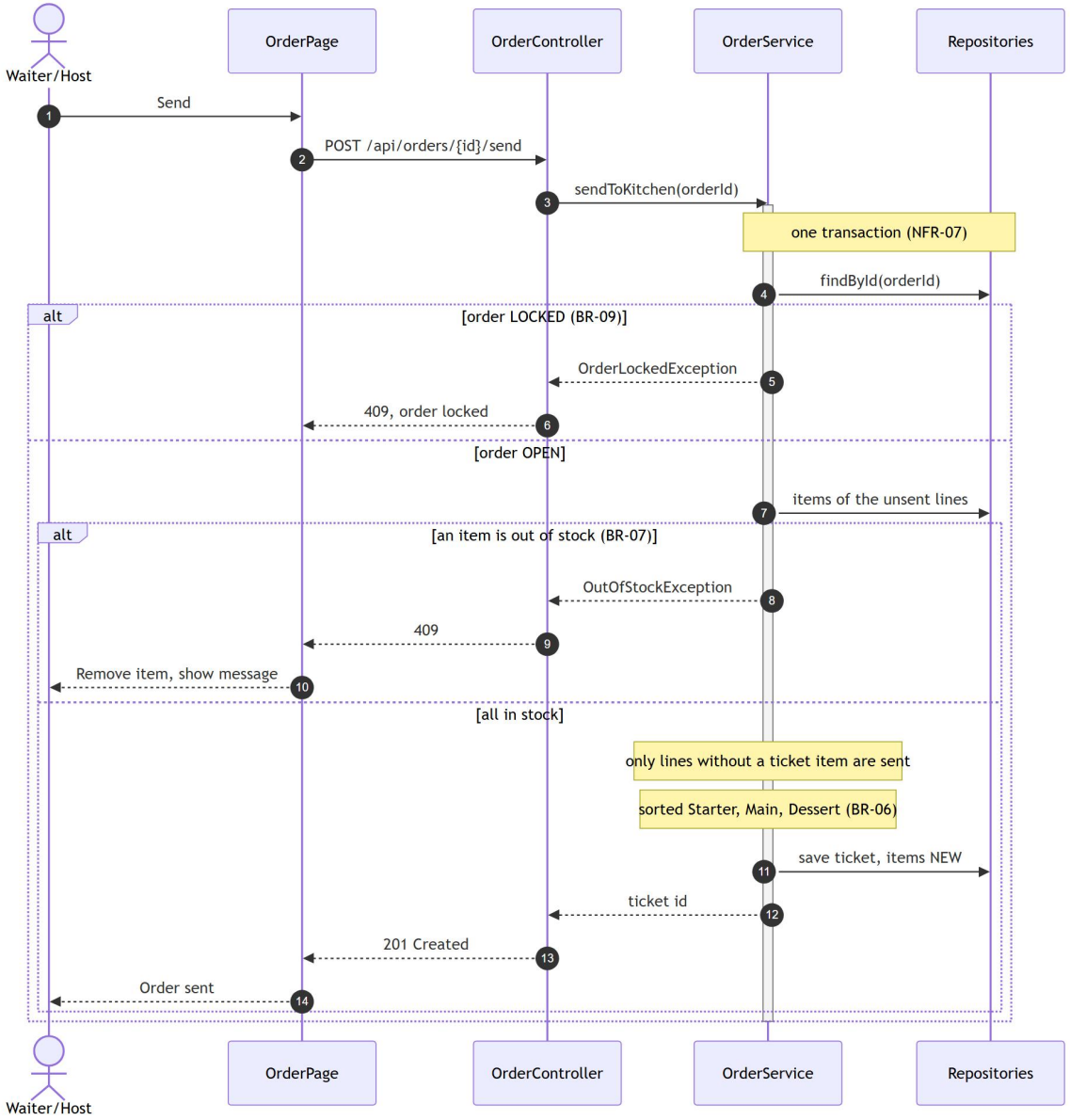


Figure C.11. Sequence diagram, UC-08 (part 2: send to the kitchen)

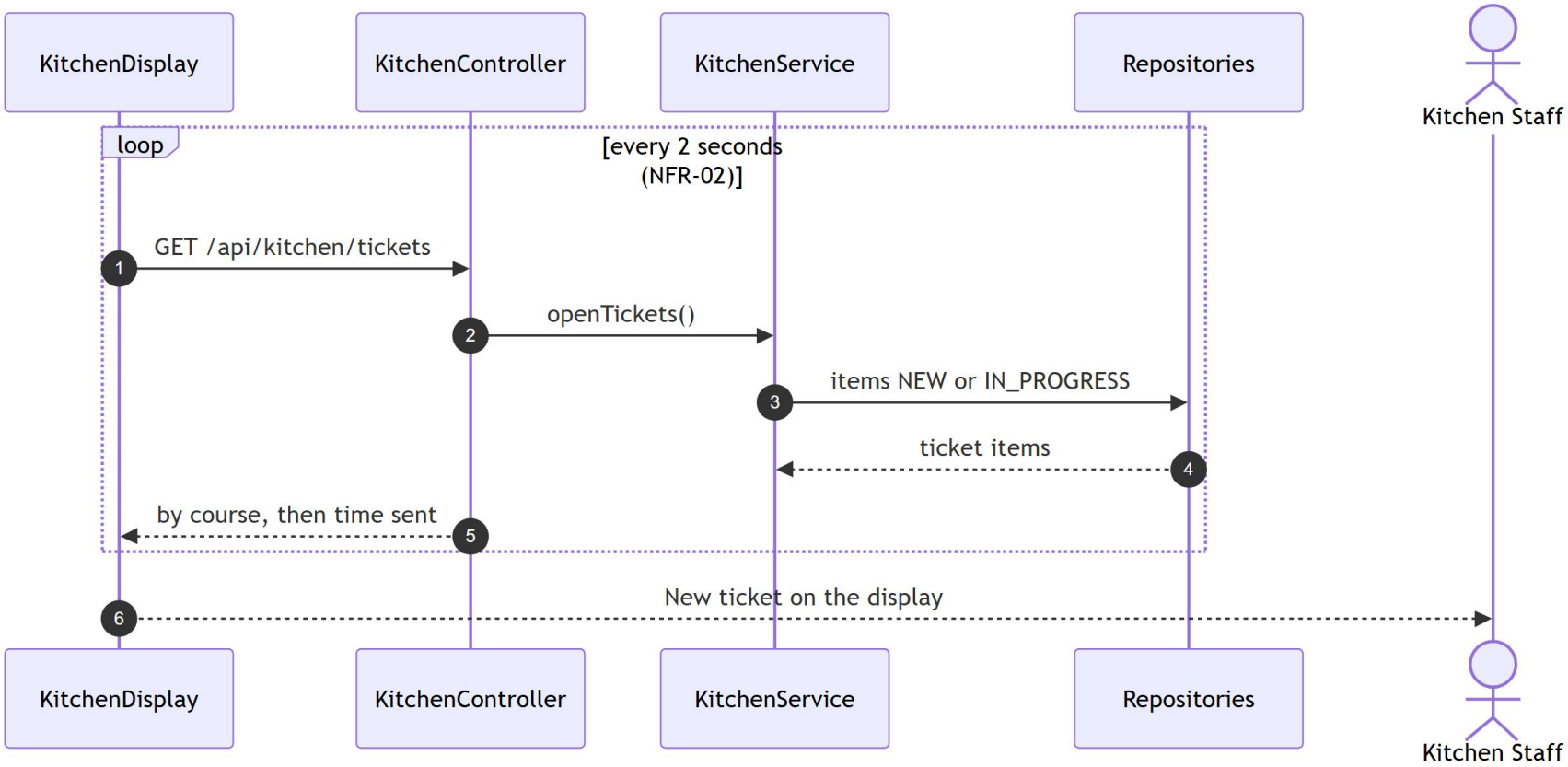


Figure C.12. Sequence diagram, UC-08 (part 3: kitchen display)

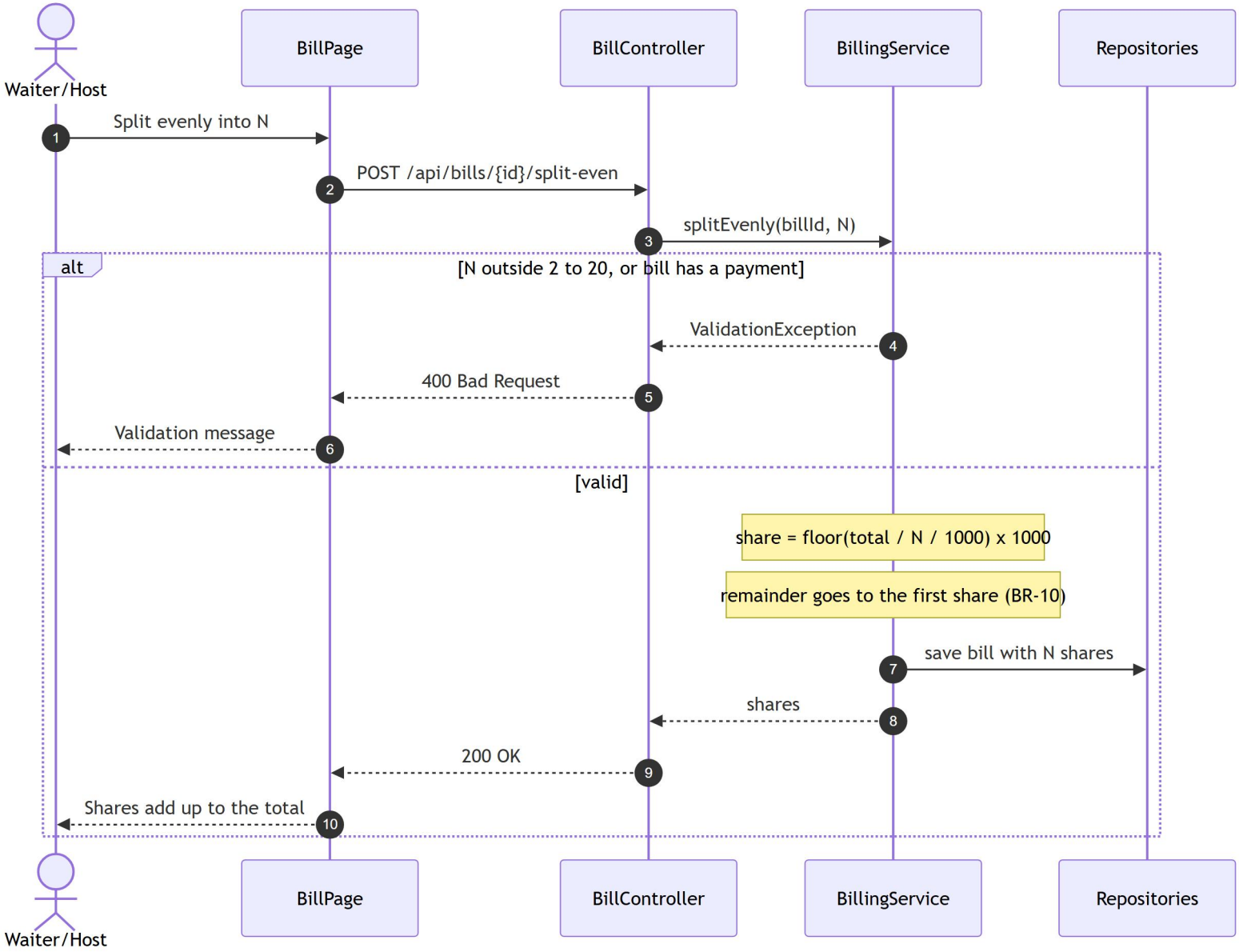


Figure C.13. Sequence diagram, UC-13 Split bill

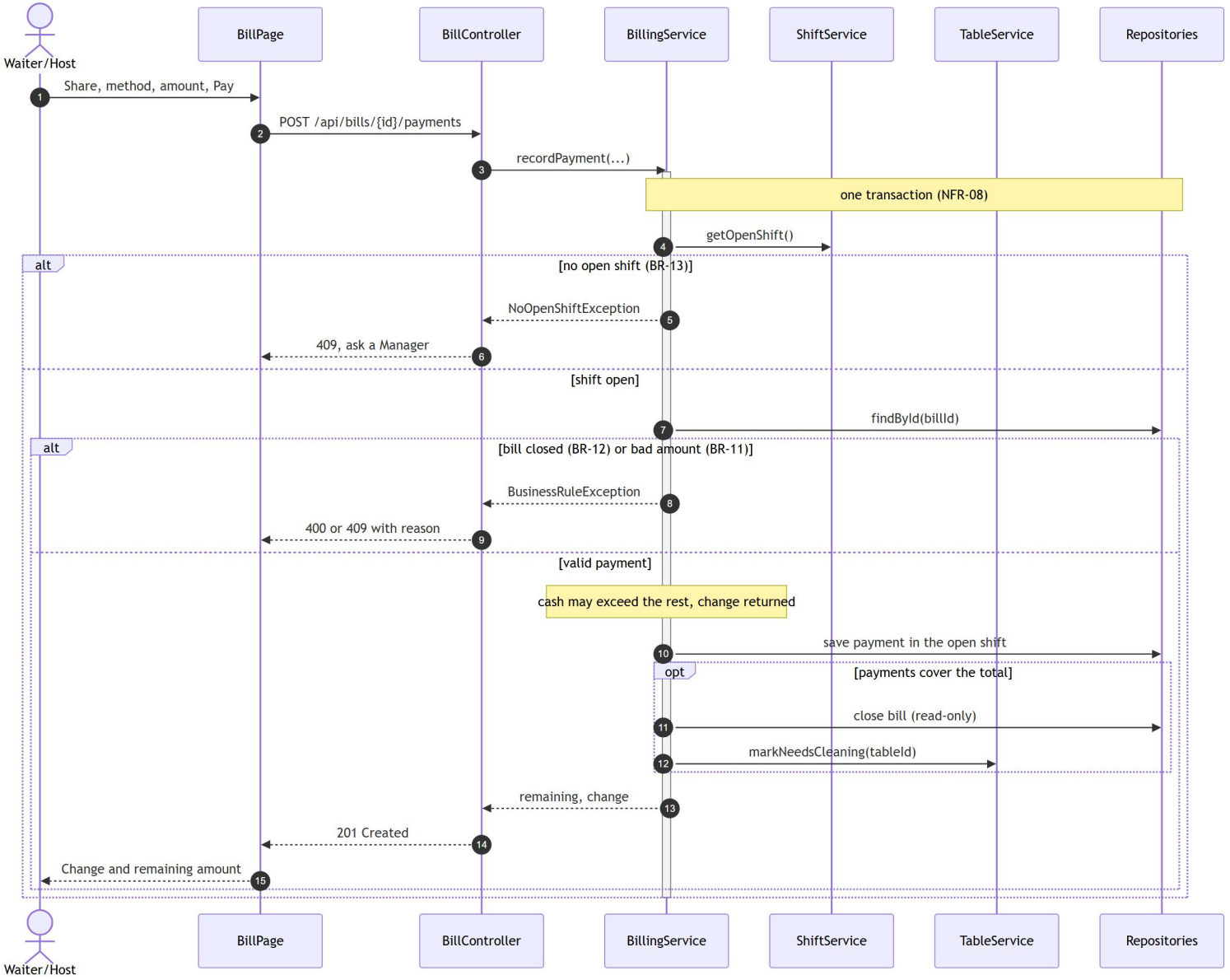


Figure C.14. Sequence diagram, UC-14 Record payment and close bill

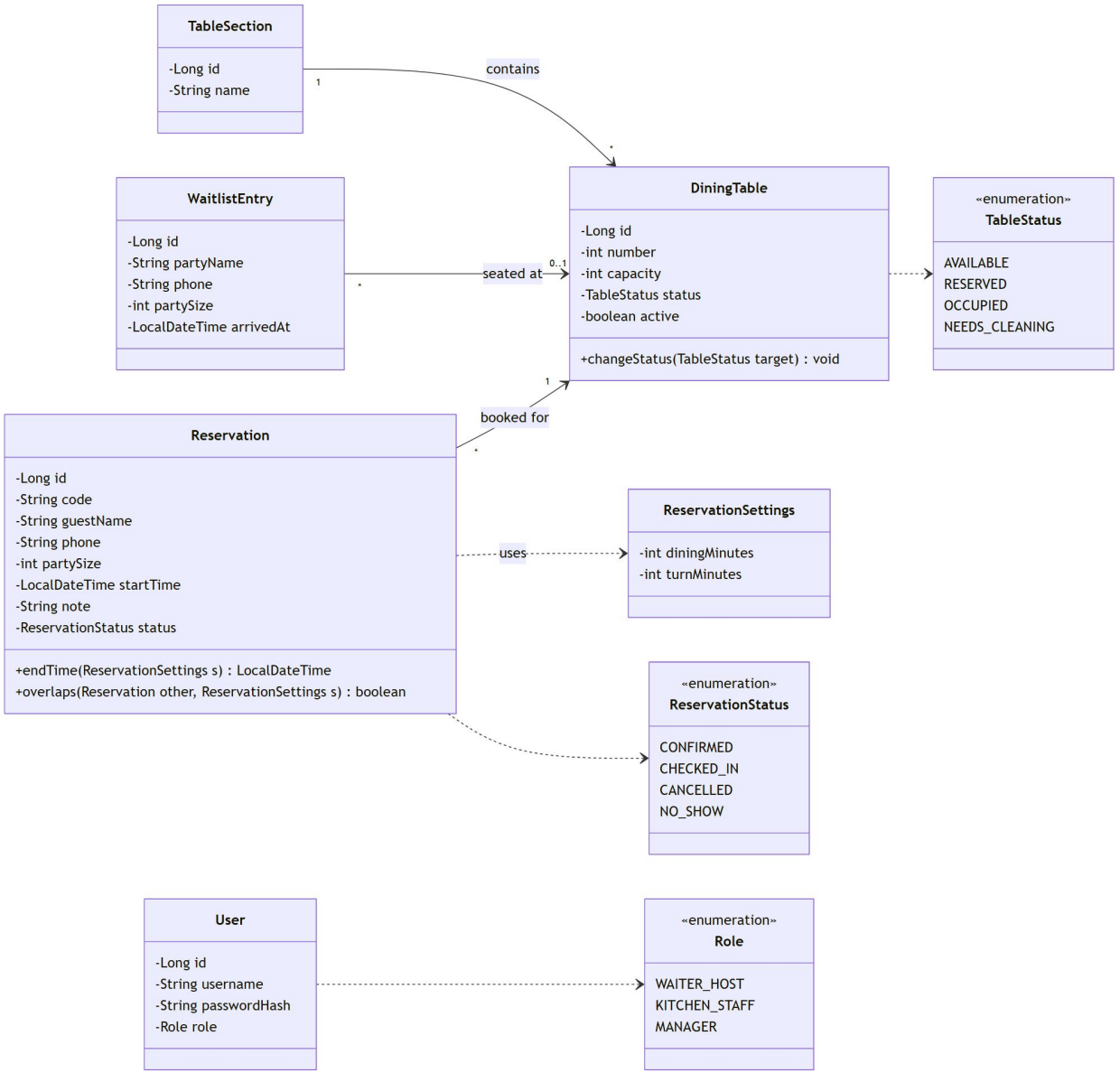


Figure C.15. Class diagram, Table & Reservation and shared classes

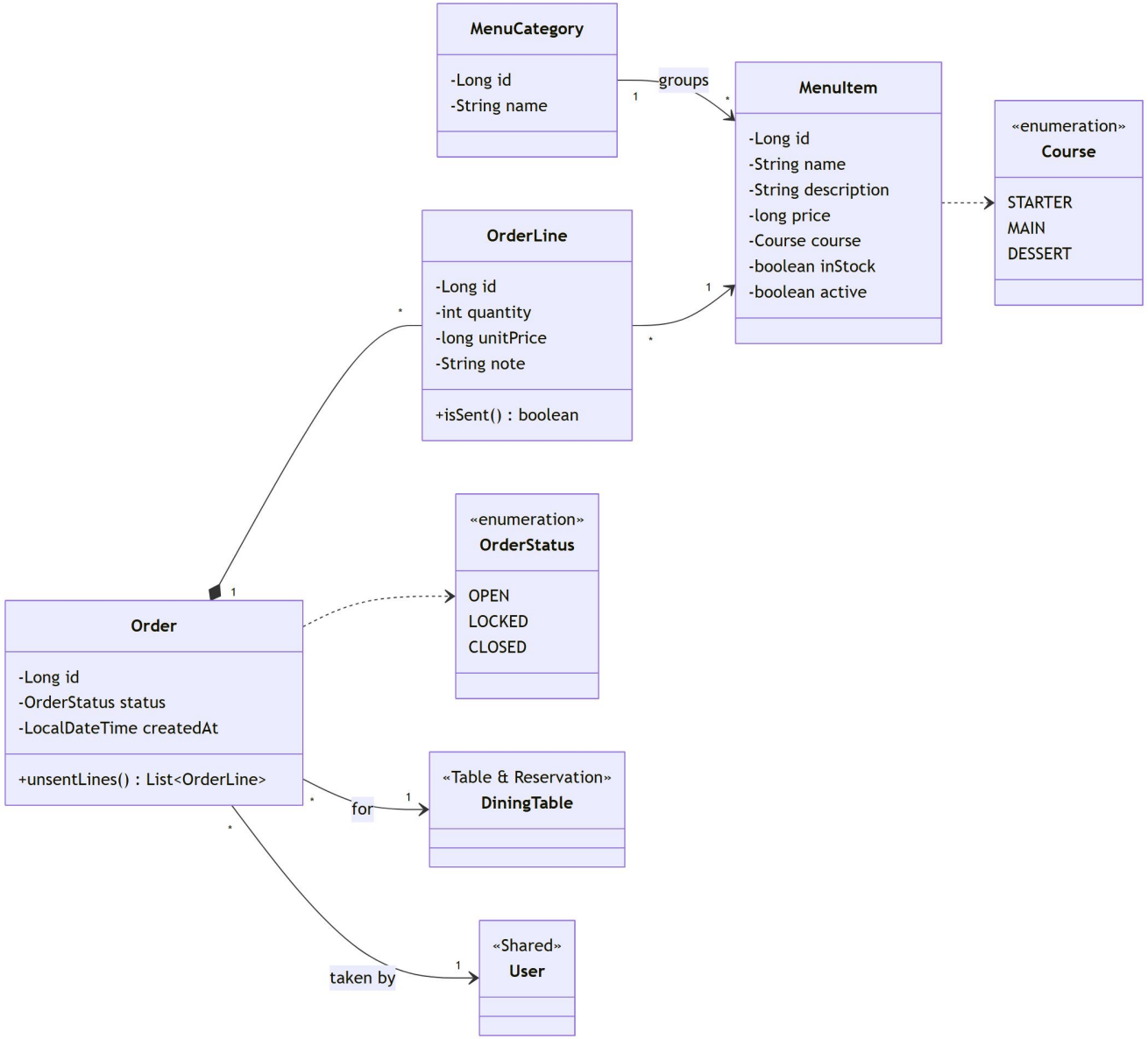


Figure C.16. Class diagram, Order & Kitchen: menu and orders

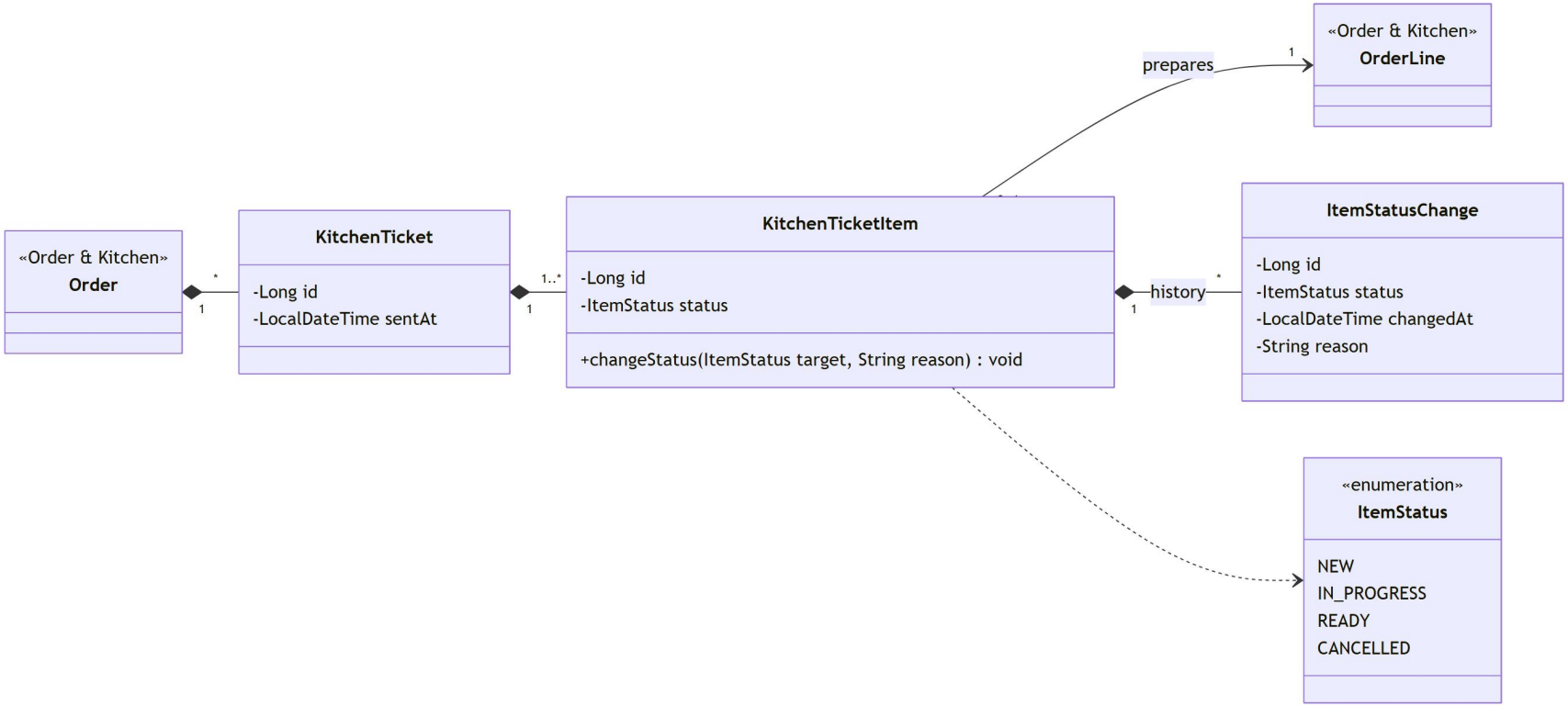


Figure C.17. Class diagram, Order & Kitchen: kitchen tickets

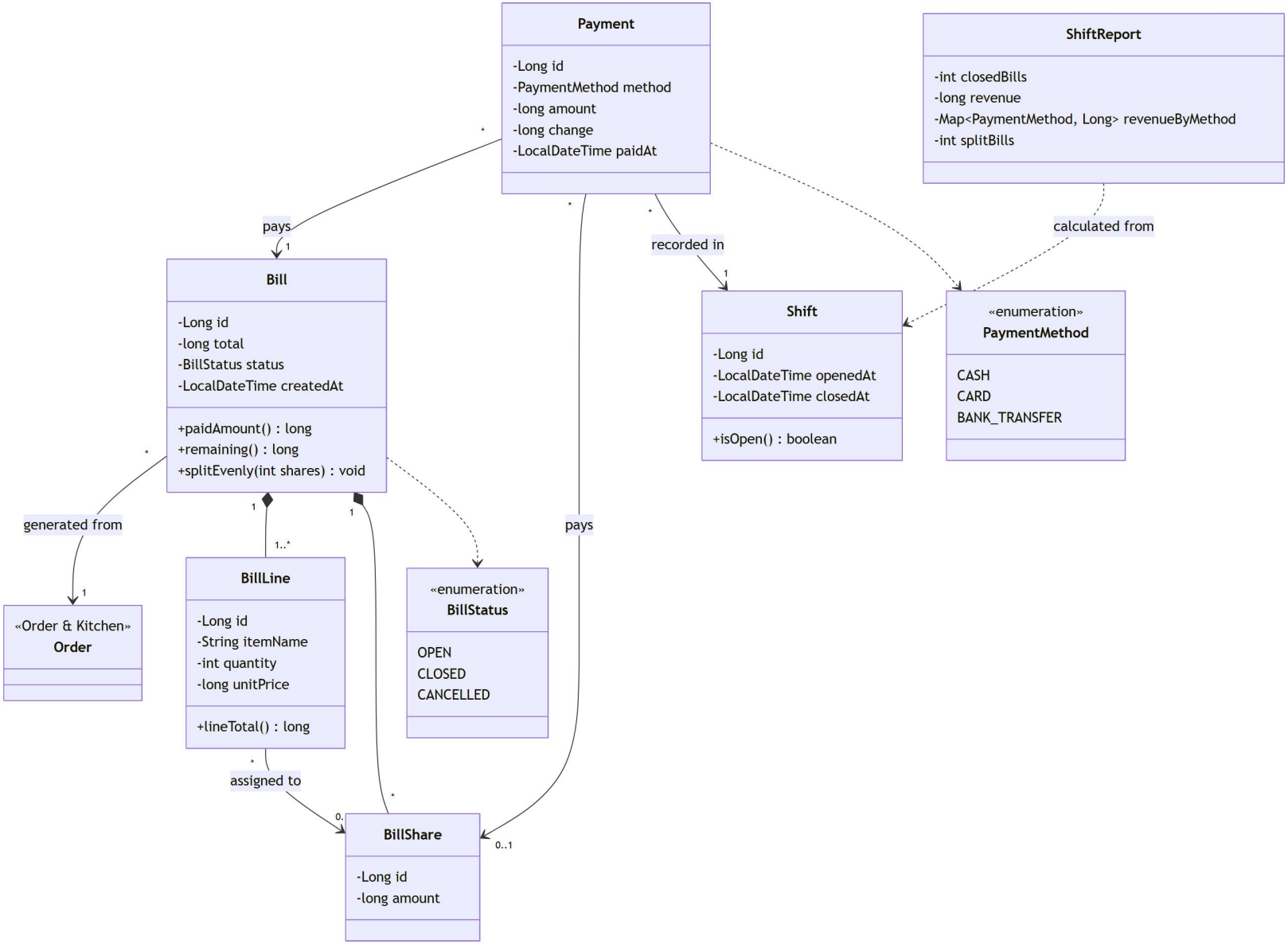


Figure C.18. Class diagram, Billing & Reporting

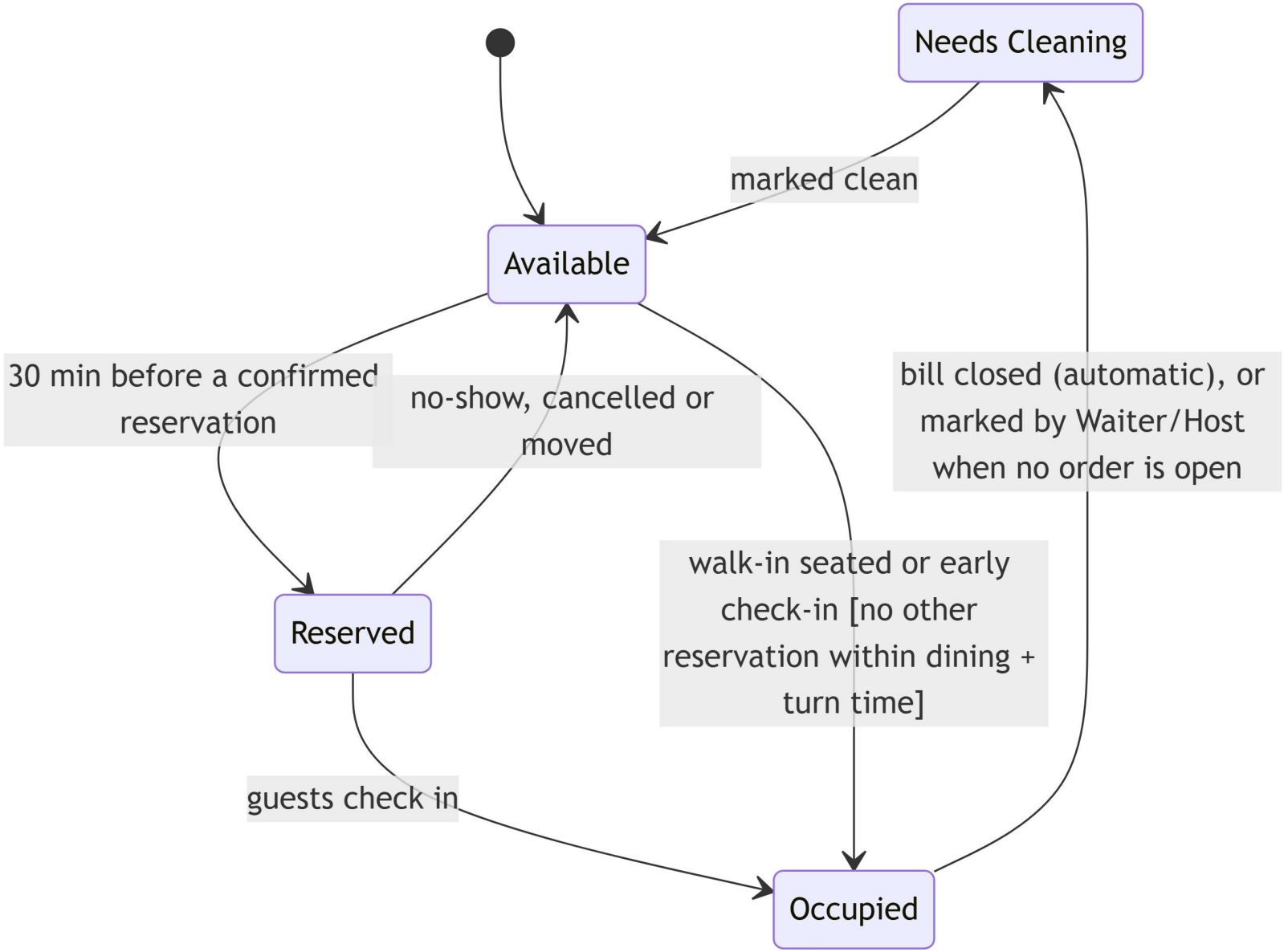


Figure C.19. State diagram, table status lifecycle (BR-05)

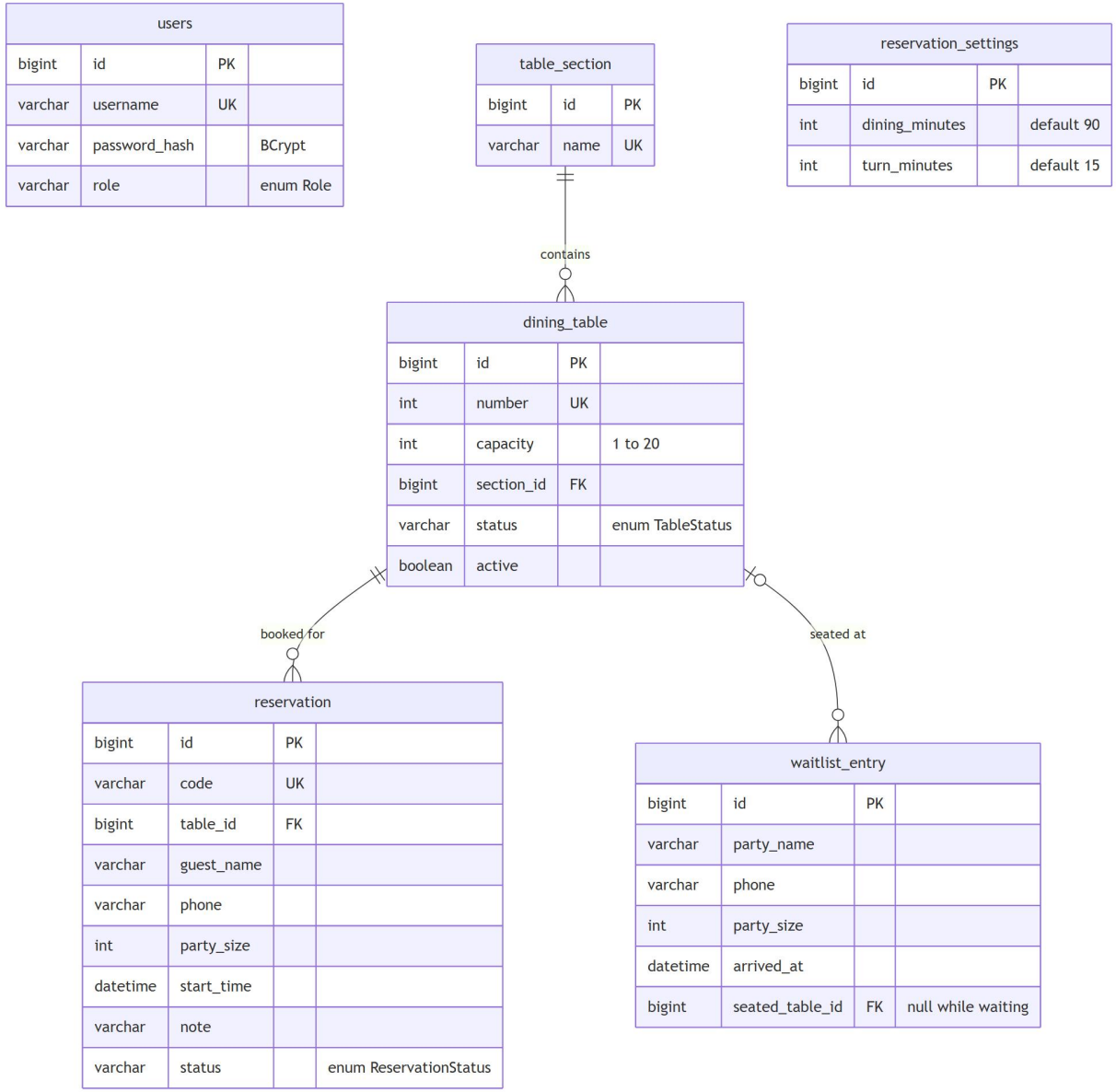


Figure C.20. ERD, Table & Reservation

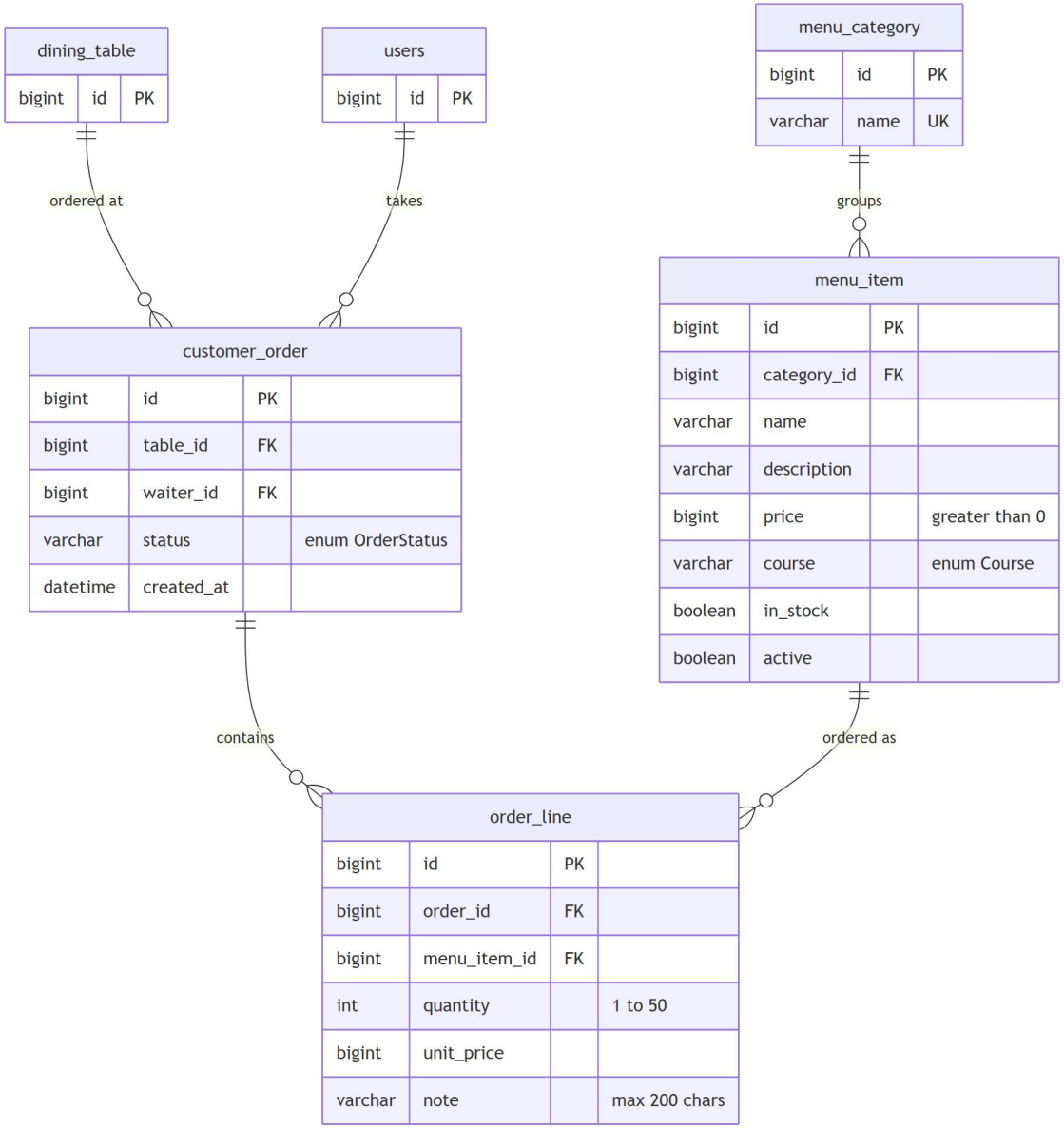


Figure C.21. ERD, Order & Kitchen: menu and orders

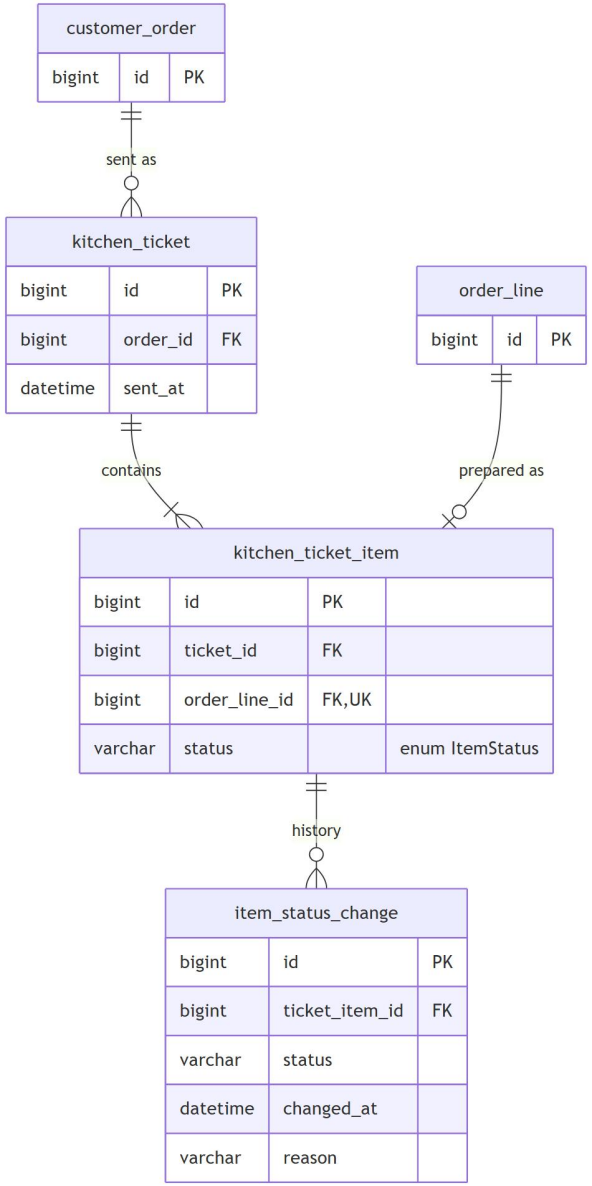


Figure C.22. ERD, Order & Kitchen: kitchen tickets

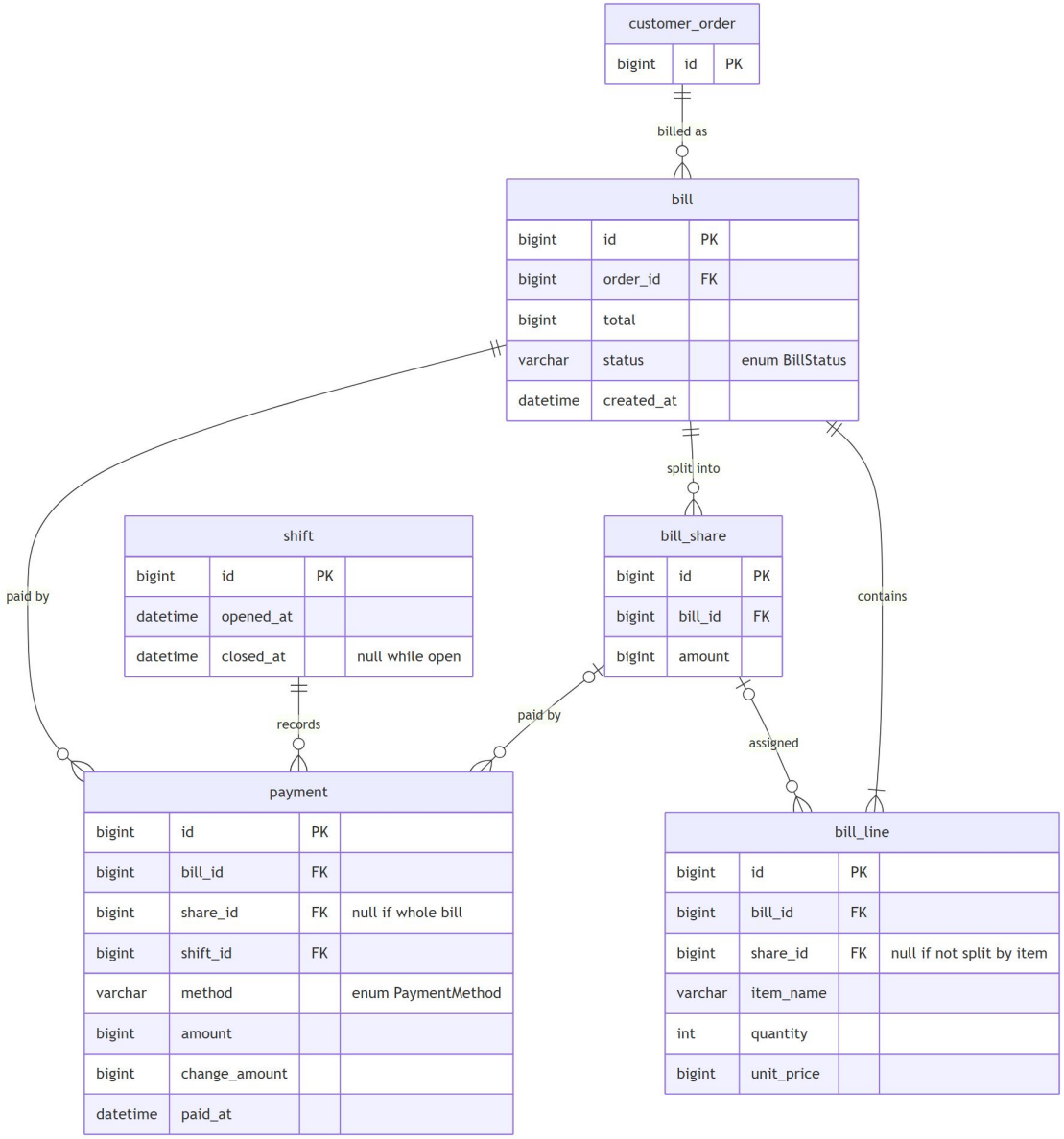


Figure C.23. ERD, Billing & Reporting

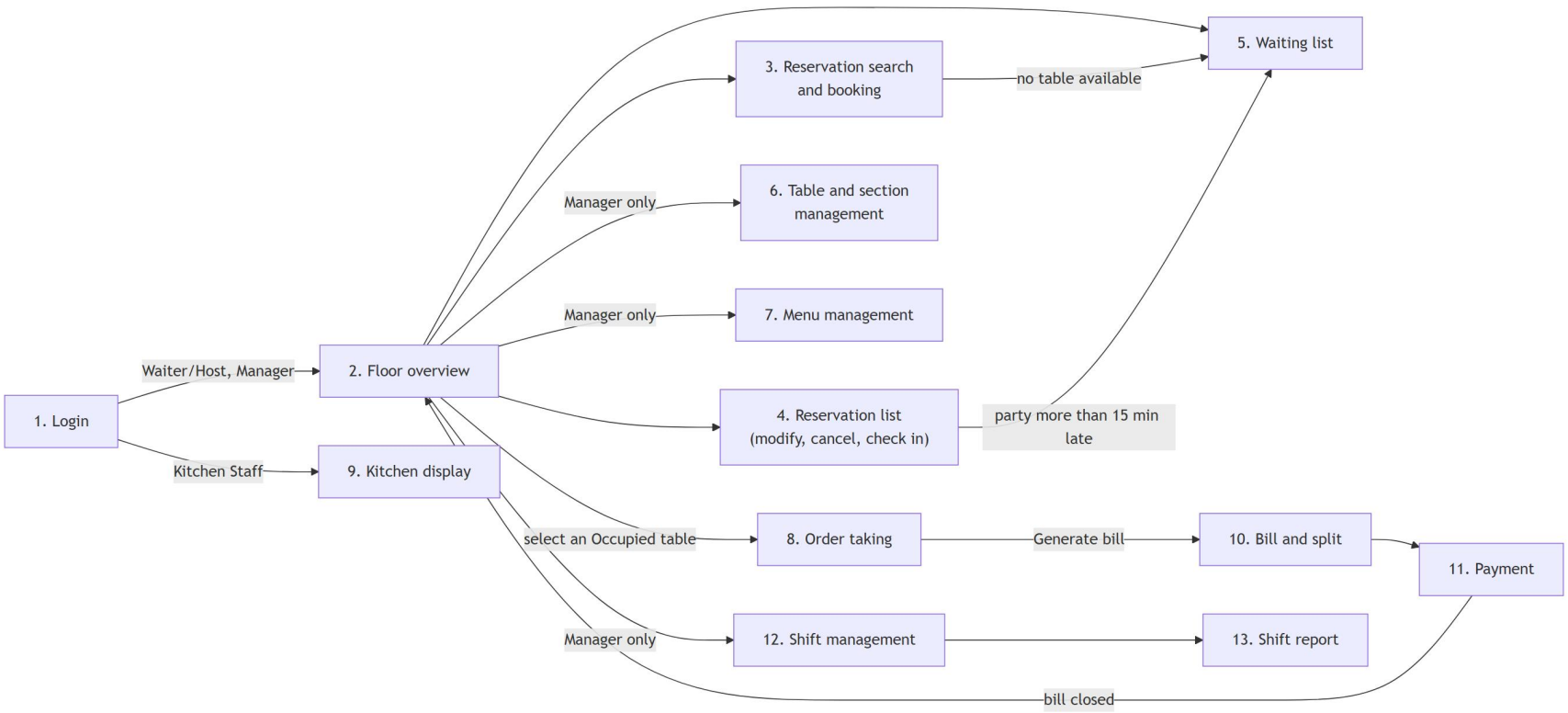


Figure C.24. UI flow, screen navigation

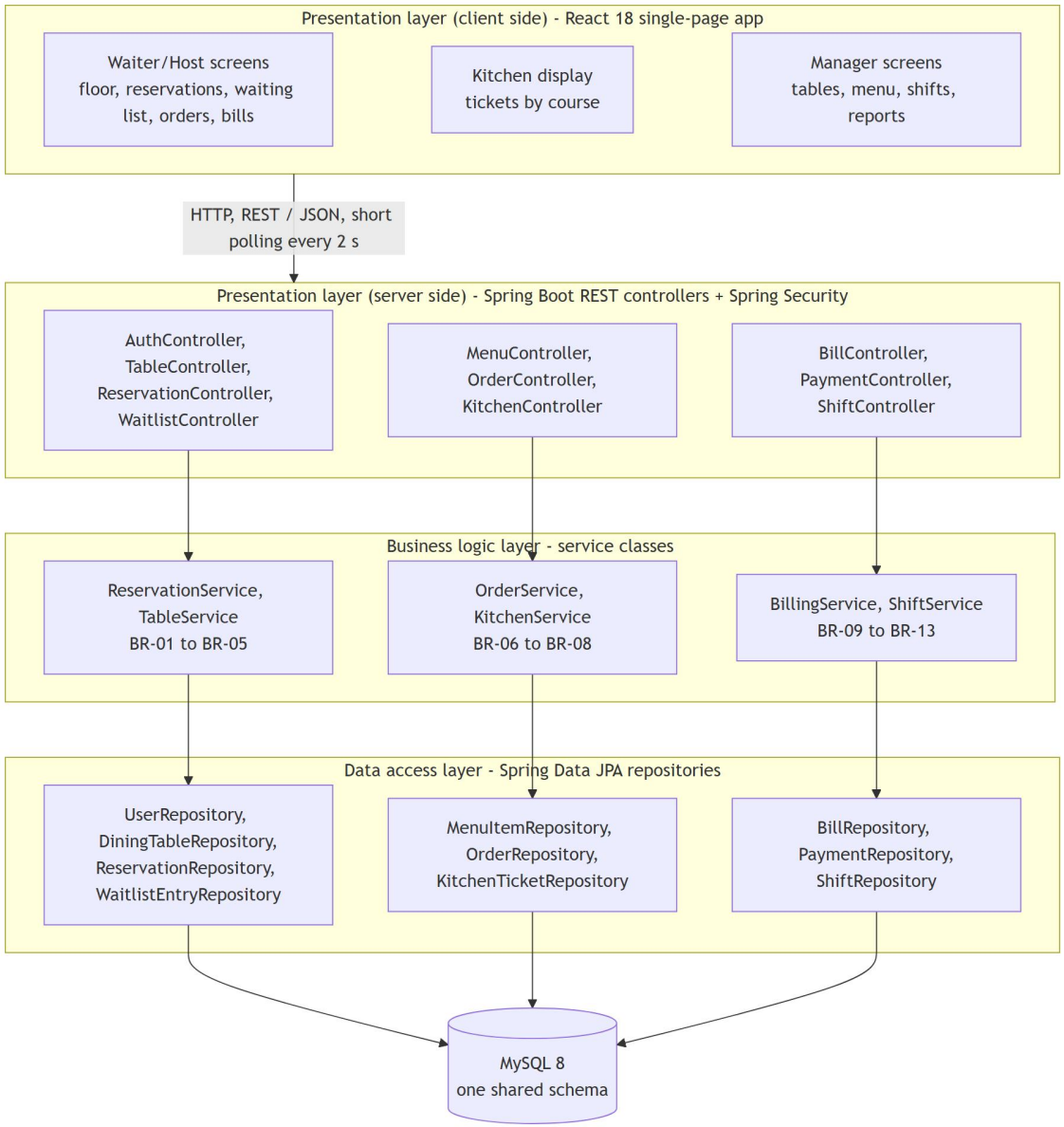


Figure C.25. Layered architecture

REFERENCES

[1] N. T. Hai, "Application Programming Project — Course Guidelines for Students," Faculty of Information Technology, Van Lang University, Ho Chi Minh City, Vietnam, 2026.

[2] N. T. Hai, "Application Programming Project — Suggested Project Titles," Faculty of Information Technology, Van Lang University, Ho Chi Minh City, Vietnam, 2026.

[3] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.

[4] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[5] Object Management Group, "OMG Unified Modeling Language (OMG UML), Version 2.5.1," Dec. 2017.

[6] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[7] A. Cockburn, *Writing Effective Use Cases*. Boston, MA, USA: Addison-Wesley, 2001.

[8] "Spring Boot Reference Documentation." [Online]. Available: https://docs.spring.io/spring-boot/ (accessed Sep. 26, 2026).

[9] "React Documentation." [Online]. Available: https://react.dev/ (accessed Sep. 26, 2026).

[10] "MySQL 8.0 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/ (accessed Sep. 26, 2026).

[11] "JUnit 5 User Guide." [Online]. Available: https://junit.org/junit5/docs/current/user-guide/ (accessed Sep. 26, 2026).

[12] T. S. M. Tse and Y.-T. Poon, "Modeling no-shows, cancellations, overbooking, and walk-ins in restaurant revenue management," *Journal of Foodservice Business Research*, vol. 20, no. 2, pp. 127–145, 2017, doi: 10.1080/15378020.2016.1198626.